Secret-Carrying Puzzles and Garbled Circuits Optimized for Zero-knowledge Proofs

Debasish Ray Chawdhuri
Manoj Prabhakaran*
draychawdhuri@cse.iitb.ac.in
manojmp@iitb.ac.in
Indian Institute of Technology Bombay
Mumbai, Maharashtra, India

Abstract

In this work, we introduce the concept of Obliviously Checkable Secret-Carrying Puzzles (OxSP) and build proof-friendly Garbled Circuits (GCs) to enable their practical implementation. OxSPs allow one to publicly pose puzzles and verify purported solutions received in response, keeping the desired parts of the puzzles and the responses hidden.

We show how OxSPs can be based on Garbled Circuits (GCs). However, this requires ZK-SNARK proofs of correctness of garbling. We note that combining existing GC and ZK-SNARK constructions results in very large computational costs for the OxSP solvers. Our main technical contribution is to design a new proof-friendly GC construction which cuts down the cost of generating a proof of correct garbling to almost a third, without resorting to non-standard cryptographic assumptions.

Beyond its use in OxSP, we expect our proof-friendly GCs to be of significant independent interest, as a tool for auditable secure 2-party computation.

1 Introduction

Many transactions carried out over a blockchain, possibly using smart contracts, are between two parties. We introduce a general purpose tool, called Obliviously Checkable Secret-carrying Puzzles (OxSP), which enhances the versatility of such transactions, by leveraging the power of secure 2-party computation (2PC).

Consider a smart-contract based transaction of the following general form: Alice offers to transfer a certain amount of crypto-currency to anyone who can post a message that satisfies certain conditions she has specified. (We shall refer to this as a puzzle). If later on, Bob posts such a message (which we shall refer to as a solution), the block-chain (implemented by a crowd with a large enough fraction of honest/rational players) automatically transfers the promised amount from Alice's account to Bob's account, without any intervention from Alice. Compared to using smart-contracts directly in such a situation, using an OxSP (which we shall elaborate on shortly) offers a few advantages:

- Privacy: External observers do not see the puzzle's solution. Even Alice does not see the solution directly, but only a desired function of the solution. Further some aspects of the puzzle itself could be kept hidden from the observers and Bob, if desired.
- Efficiency: Without using any off-chain mechanisms, the observers' effort can be small and independent of the size of the challenge or the solution, unless a dispute arises between the two transacting parties. Such disputes are irrational as they can be publicly resolved (with a bit of effort from the observers) and penalties can be attached to false claims.

We describe a few example use cases to demonstrate our motivation. Consider Alice looking to *buy* a solution for a hard optimization problem (e.g., an instance of the Traveling Salesperson Problem, Integer Linear Programming, or Max-SAT, or a cryptographic problem like finding a small pre-image for a hash function). Suppose she already has one candidate solution, and she is interested in purchasing a better solution. If she were to directly use a smart-contract for soliciting such a solution, she will need to post her solution (or at least its value) to ensure that only better solutions will get paid; further, someone wishing to transact with Alice will have to reveal their solution publicly rather than just to her.

With an OxSP, Alice can post a challenge which hides her solution; then anyone can attempt to sell her a solution without revealing it to anyone else, and further it is not revealed to Alice either, unless it meets the criterion she has set. In all of this, the observers need not take any action till a time-limit, when the purchase will be finalized (seller getting the promised amount) or Alice declares, after obliviously checking the submitted solution, that it is not a better solution than her own. In the latter case, she will be able to prove this to the seller and the public, while keeping both solutions secret; furthermore, verifying this proof will be very efficient (involving only one hash evaluation).

As one may expect, Zero-Knowledge (ZK) proofs will play a role in implementing OxSP. However, ZK proofs are not enough for OxSP. Indeed, for Alice and Bob to check if the submitted solution is better than hers when neither party knows both of the solutions, what is required is an instance of 2PC. As we shall see, ZK proofs combined with a classical tool for 2PC, namely Garbled Circuits (GCs), will yield a viable implementation of OxSP. Apart from conceptualizing OxSP, the key technical challenge we attack is of reducing the cost of constructing ZK proofs about Garbled Circuits.

OxSP is powerful enough to support more sophisticated applications. Below, we illustrate a few such applications:

- **Smart Contracts with Secret-Handshakes:** OxSP can be used to create smart contracts between a pair of parties such that the contract is engaged only when secret parameters supplied by the two parties "agree" with each other (e.g., the asking-price by a seller is no more than the price that a potential buyer is willing to pay; parameters of a product from a potential supplier meet the buyer's requirements; credentials supplied match credentials sought; etc.). More generally, the terms of the contract

can be determined by the secret parameters (e.g., if a buyer provides a valid discount coupon, a pseudonymous identifier for the coupon is revealed to prevent double-spending of coupons, while transferring only the discounted price to the seller).

- **Private Auditing of Smart Contracts:** In a scenario as above where smart contracts carry secrets, they may need to be audited for compliance with laws and/or for fairness. By encoding a smart contract as an OxSP, statistical tests (with secret test-cases, encoded as OxSP solutions) can be performed on it, with publicly verifiable outcomes.
- **Private Benchmarks for (Small) ML Models:** Many widely used traditional ML models (like Random Decision Forests used in banking and financial industrial sectors, or decision logic for fraud detection or spam filters) can often be implemented as circuits with fewer than a million gates. By encoding test data as secrets in an OxSP, and model parameters as secrets in a solution, a public leader-board can be maintained without revealing these secrets.
- In the above applications, one can add an element of trustworthy *randomization* (e.g., for lotteries when choosing one of many qualifying bids, or for randomizing statistical tests or test-cases) without resorting to an external source of randomness. For this, the puzzle's secret can incorporate a key for a pseudorandom function.

Related Work

Our work lies at the intersection of two slightly different lines of work: bringing blockchains to secure multi-party computation (MPC) and bringing MPC to blockchains. In the former case, the focus is on leveraging the features of blockchains, like smart contracts, digital assets, and a bulletin-board to enhance the efficiency or fairness of MPC. The latter line of work, which we mainly fall into, focuses on using MPC techniques to enhance the efficiency and features of blockchains.

Blockchains for MPC. While OxSP is best viewed as a tool for blockchains, it does retain the functionality of general-purpose 2PC (when "private feedback" is also incorporated, as discussed later). Hence, one may view our work as also continuing the thread of leveraging blockchain features to enhance 2PC. In this perspective, the main features that OxSP brings to 2PC are public auditability and fairness, in a minimally interactive setting.

Blockchain has been used for public auditability of MPC protocols [11, 24, 28, 31, 32]. In particular, [11] describes a protocol called GABLE that uses garbled circuits to hide the details of computation and enable MPC over Ethereum blockchain, assuming a trusted garbler. [32] uses garbled circuits over blockchain to achieve the same, settling for a covert security model, which allows them to use a relatively light-weight cut-and-choose protocol. [28] focuses on cheater detection to ensure fairness. [24] also uses a blockchain to enable public auditability to an MPC protocol.

Fairness is a related MPC feature enabled by blockchains. In the context of MPC, fairness refers to the requirement that if any party receieves an output from a protocol, all honest parties must too. Motivated by the fact that without honest majority, fairness is impossible to achieve in general purpose MPC, the possibility of using external setups has been studied. In recent years, blockchain models, with their smart-contract features, have emerged as a popular framework for enabling fairness in 2PC and MPC [6, 15]. The general idea in these constructions is to detect and penalize a party in an MPC protocol if they do not cooperate in allowing all players to obtain outputs. For this the parties would deposit cryptocurrencies before starting an MPC protocol (with public auditability/cheater detection features) that is carried out over a blockchain.

MPC for Blockchains. Much of the work in this line has focused on a very special case of 2PC, namely Zero-Knowledge (ZK) proofs (see e.g., [29]). This is primarily due to the remarkable improvement of efficiency of ZK proofs, in the form of Succinct Non-Interactive Arguments of Knowledge (SNARKs) and related technologies. In the current work, we leverage these tools, but go beyond the functionality of ZK. As discussed earlier, OxSP enables several new blockchain applications.

Non-Interactive Secure Computation and Relatives. Another theme of this work is to enable 2PC with security against active corruption in a non-interactive manner. This problem and its variants have been extensively studied, particularly in the form of Non-Interactive Secure Computation (NISC) [1, 10, 20, 22, 23].

GCs form a natural tool for NISC. However, traditionally, using a ZK proof for proving the correctness of a GC has been considered unrealistically inefficient, and much work has focused on alternate techniques like cut-and-choose, MPC-in-the-head, and customized ZK protocols exploiting features of specially constructed GCs [3, 4, 21, 25, 26]. Much of this work has focused on asymptotic efficiency, and are still not practical.

However, with the advent of efficient ZK-SNARKs, using them for NISC has been explored. In particular, [1] shows that using the hash-based Ligero proof system [5] to prove the correctness of GCs can lead to a reasonably practical NISC protocol. Our construction could be viewed as an alternative that uses the *more widely adopted* proof systems based on elliptic-curve cryptography (ECC), thereby making it easier to integrate into existing blockchain eco-system. Further, any concrete efficiency improvements in the ECC-based SNARK systems will be directly exploitable in our construction.

However, we remark that for large puzzles, using a hash-based proof system to instantiate OxSP could be advantageous. An exact comparison between the costs in our construction and one using the approach of [1] is difficult, due to the very different proof systems employed. Nevertheless, just as in the case of different proof systems having their own merits, we believe that the different trade-offs offered by these two approaches will be of interest to different applications of OxSP.

1.1 Our Results

Our contributions are two-fold:

- We put forward OxSP as a versatile primitive for use in blockchains, and show that it can be efficiently instantiated using a Garbled Circuit (GC) construction that supports efficient proof of correctness, and an appropriate Oblivious Transfer protocol.
- We then present a novel GC construction using elliptic-curve cryptography that turns out to be significantly more efficient to

generate *succinct* correctness proofs for, compared to the (otherwise more efficient) GC constructions based on symmetric-key cryptographic primitives.

While GC constructions have traditionally exploited fast symmetric-key primitives and have achieved remarkable efficiency levels (see e.g., [19]), their use in many situations is hampered by their restriction to a semi-honest adversary model. Our approach to addressing this is to allow basing GC on elliptic-curve primitives (we rely on the DDH assumption). Using elliptic-curve primitives is more expensive in terms of constructing the GC itself, but leads to much better efficiency in the more expensive step of proving the correctness of the GC.

Alternatives. We also explore alternatives for proof-friendly GC constructions using proof-friendly symmetric-key primitives. One possibility (though not readily available in the literature) is to use lookups for proving the correctness of a standard AES-based GC; we find that this remains quite expensive. Another possibility is to use the Poseidon 2 hash function [17] as a random oracle in a traditional GC construction. We study this alternative and propose a new GC construction relying on a proof-friendly instantiation of a random oracle; as opposed to the baseline (using Poseidon 2 as random oracle in a conventional GC construction), this achieves efficiency comparable to our main construction. However, relying on a relatively untested and highly structured hash function as a random oracle for the privacy of the GC itself (rather than just the soundness of ZK proofs) is arguably undesirable, and hence we consider our main construction as dominating this alternative. Nevertheless, this construction maybe of interest and is described in Appendix B.

Evaluation. The main cost in the system is the computational effort of the Solver in generating proofs of correctness of garbling. In ECC-based ZK-SNARKs, this cost is directly accounted for by the number of gates in the circuit for the statement that is proven. Hence, for a fairly accurate comparison of our construction with alternative approaches using ECC-based ZK-SNARKs, we can simply count the number of these gates. Table 1 shows this comparison. We estimate that in a well-provisioned system (e.g., with a gaming GPU and 64 GB RAM), using currently available GPU implementations of compatible ZK-SNARK systems (specifically, UltraPlonk based on [13, 14]), this translates to a few seconds of Solver computation time for every few thousand gates in the puzzle circuit.

1.2 Technical Overview

OxSP from Garbled Circuits and ZK-SNARKs. An OxSP challenge is specified by a function $P(x, \cdot)$, where P is a public function and x is a secret component of the puzzle. A solution y can be evaluated using this function to obtain a "grade" for the solution, as well as additional optional information about the solution that is revealed to the challenger.

Since OxSP involves an arbitrary computation (namely, the evaluation of a puzzle) with secret inputs, it is natural that tools from secure 2PC can be leveraged to implement them. As mentioned earlier, the tool of choice for us is Garbled Circuits (GCs).

First, we note that a GC is a "one-time" object: it can be securely evaluated on only one input. On the other hand, a single puzzle posted by a challenger must be available to all the solvers. Hence, the puzzle itself cannot be implemented using GCs. Instead, we shall use GCs *to encode the solutions* to a puzzle: each solver will post a GC implementation of the puzzle circuit, with its solution included as input, and this solution will be evaluated by the challenger. The grades, as well as any information about the solution that must be revealed to the challenger, will be produced by this evaluation process; as explained later, the grades will be encoded in a manner that is easy to publicly authenticate.

To prevent the solver from cheating by altering the puzzle, it is important that the solution includes a proof of correctness of the GC. This leads us to our key technical challenge, which we shall shortly address.

There is one more aspect of our OxSP construction that needs explanation, namely how the secret x is encoded in the puzzle. Following how a GC is used in 2PC, we employ an Oblivious Transfer (OT) protocol to enable the challenger to get its secret input x encoded using the labels of the GC for each solution. We rely on a UC-secure 2-round OT protocol, using a CRS as a setup [27]. In such a protocol, the first message is generated by the receiver whose input is a "choice bit" b, and the sender whose input is a pair of strings (s_0, s_1), generates the second message; the receiver can then recover s_b from the second message, with help of the state information it has retained from the computation of the first message. In our OxSP construction, to encode each bit of x, the challenger generates the receiver's message using that bit as the choice-bit. These first OT messages which encode x, combined with a puzzle circuit (in the clear) forms the full challenge. The puzzle solution will include a garbled circuit encoding the puzzle circuit, OT second messages that allow the challenger to recover the labels for the wires corresponding to x, and also the labels that encode the actual solution y.

ZK-Friendly Garbled Circuits. A standard "gate-by-gate" garbled circuit construction involves picking a "label" for each wire of the circuit, and designing "garbled gates." Given one label each for the two input wires of a gate (say $\mathcal{K}_u^a$ and $\mathcal{K}_v^b$, where u, v are the indices of the wires and $a, b \in \{0, 1\}$), the garbled gate allows the evaluator to recover the correct label for the output wire of that gate (i.e., $\mathcal{K}_w^{g(a,b)}$, where w is the output wire index and g is the gate's function).

As such, a garbled gate implements a function $\mathbf{\Psi}$ that maps two labels to another label. We shall refer to the garbled gate by this function $\mathbf{\Psi}$ through out the sequel.

Our approach to implementing a garbled gate is to rely on elliptic-curve cryptography, as popular SNARK systems can natively support computations over many elliptic curves.

Given an elliptic-curve group $\mathbb{G}$ over a field $\mathbb{F}_q$ with prime-order p, where DDH is hard, and two random non-identity elements $\mathbf{g}, \mathbf{h} \in \mathbb{G}$, the map $\mathcal{K} \mapsto (\mathbf{g}^{\mathcal{K}}, \mathbf{h}^{\mathcal{K}})$ serves as a pseudorandom generator that yields two group elements from a single label (in $\mathbb{F}_q$, but effectively in $\mathbb{F}_p$).[1] We shall rely on a simple function $\mathrm{x} : \mathbb{G} \to \mathbb{F}_q$ to map a group element back to an element in $\mathbb{F}_q$. The garbled gate function $\mathbf{\Psi}$ (which maps input labels to an output label, as mentioned above)

[1] To allow fan-out, multiple such $(\mathbf{g}, \mathbf{h})$ pairs will be used.

Scheme	No. of gates involved in the proof per garbled gate (approx.)		
	Addition	Multiplication	Lookup
SHA-256 based [16]	88,000	368,000	0
AES-based (using [2])	360,000	40,000	0
AES-based with lookups (new)	0	0	4,848 (7 lookup wires)
Our main scheme with lookups	332	896	128 (4 lookup wires)
Alternative scheme using Poseidon-2 (Appendix B)	1,280	1,024	0

Table 1: Cost of generating proof of correctness per garbled gate, in terms of the number of arithmetic gates in the relation being proven.

is implemented using a polynomial P of four variables over the field $\mathbb{F}_q$:

$$\Psi(\mathcal{K}_u, \mathcal{K}_v) = P\left(\mathrm{x}(\mathbf{g}^{\mathcal{K}_u}), \mathrm{x}(\mathbf{h}^{\mathcal{K}_u}), \mathrm{x}(\mathbf{g}^{\mathcal{K}_v}), \mathrm{x}(\mathbf{h}^{\mathcal{K}_v})\right).$$

The monomials in P (there are 12 of them in our construction) are chosen such that, given input and output labels for Ψ along with the constraints of a binary gate, the coefficients of P can be sampled so as to let Ψ satisfy the constraints, and so that, under the DDH assumption, this yields a secure garbled circuit.

A SNARK to prove the correctness of garbling may use "lookup gates" to represent the computation of elliptic curve exponentiation using a relatively small number of gates. This uses precomputed certain exponents of the group elements $\mathbf{g}$ and $\mathbf{h}$. As we shall see, it is also required that the pair $(\mathbf{g}, \mathbf{h})$ needs to vary across "neighboring gates" (i.e. gates that share at least one of the inputs). Combined, these two requirements impose an *a priori* limit on the fan-out, chosen at the setup (though fan-outs larger than this limit can be supported by pre-compiling the circuit to use additional gates to reduce the fan-out per gate).

Proving Security of Garbling Schemes. Our garbling scheme, as well as the classical ones, derive their security from the construction of each gate. A contribution of this work is to formalize this intuition by giving a security definition for garbling a single gate, so that when this construction is used in a specified manner for garbling an entire circuit, the resulting construction is a secure garbling scheme. We believe this could be of independent interest in the future, as it allows one to prove the security of a gate-by-gate GC construction without repeating the gate-by-gate hybrid argument.

Proving Correctness of OT. Our construction also requires that the garbler proves the correctness of the OT messages it sends. It turns out to be cheaper to disassociate this proof from the ZK-SNARK for the correctness of garbling. For this, the garbler will commit to the strings to be transferred under OT, and prove (as part of the GC correctness proof) that those strings are indeed the correct labels, and separately prove that the message sent in the OT protocol corresponds to the committed strings. For the latter, we use Bulletproof [9].

2 OxSP: Syntax and Security

An OxSP scheme allows a challenger to publicly post a puzzle, some parts of which can be kept hidden. Then puzzle solvers can post encoded versions of their solutions so that the challenger can obliviously check if the puzzle was solved (without necessarily learning the entire solution). Further, the challenger can assign a "grade" for the solution, and prove the correctness of the grade without revealing anything else about the secrets in the puzzle or the solution.

Graded Secret-Carrying Puzzle. Technically, a program P is a graded secret-carrying puzzle (GSP) if it has a simple structural property: its input has two parts and the output also has two parts. When defining an OxSP scheme for a GSP, these parts of the input and output are treated differently, according to the following interpretation. The input is of the form (x, y), where x is a "secret" that will be included in the puzzle, and y is a "solution" for the puzzle. (The secret is optional, and there are meaningful GSPs which do not have any secret components.) The output is of the form (g, v) where g is a grade for the solution and v is a "view" of the solution that is revealed to the challenger.

Definition 2.1 (OxSP: Syntax). An OxSP scheme for a family of graded secret-carrying puzzles $\mathcal{P}$ consists of five algorithms: setup, mkPzl, mkSol, evSol, chkGrd, with the following syntax.

(1) setup publishes a common reference string that is given as an implicit input to all the other algorithms.
(2) mkPzl takes as input a pair (P, x), where P is a graded secret-carrying puzzle program and x is the secret input to the puzzle. The algorithm outputs (P, Z) as the puzzle, and a private state σ.
(3) mkSol takes as input (P, Z, y), where P, Z are as above and y is an attempted solution, and outputs an encoded solution L.
(4) evSol takes as input (σ, L) (computed as above) and outputs a "grade certificate" G and an output v. If the certificate is a special value $\perp$, then v is empty; otherwise the grade certificate $G = (g, \pi)$ where g is a grade.
(5) chkGrd takes as input (P, Z, L, G) as above, and accepts or rejects the grade certificate G. If accepting, it also outputs a grade g (possibly $\perp$, if G itself is $\perp$).

The algorithms in an OxSP scheme are illustrated in Figure 1.

To describe the security of an OxSP scheme, we present an *ideal functionality* involving 3 parties: Challenger, Solver and Observer. As is conventional, we allow the ideal functionality to behave differently based on which parties are corrupt.

Ideal Functionality:

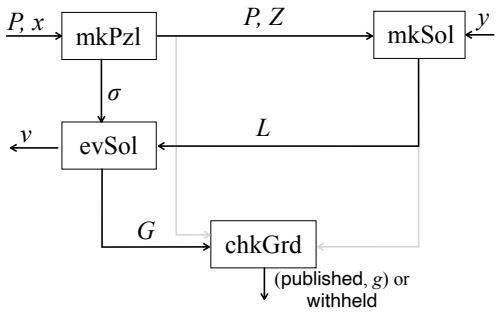

Figure 1: Components of an OxSP scheme (other than setup) described in Definition 2.1.

- Receive (P, x) from Challenger, and send P to Solver and Observer.
- Receive either $\perp$ or y from Solver.
  - In the first case, send $\perp$ to Challenger and (published, $\perp$) to Observer and terminate.
  - In the second case, proceed as follows.
- Let $(g, v) = P(x, y)$. Send (g, v) to Challenger.
- If either Challenger or Solver is honest, receive publish or withhold from Challenger. In the first case, send (published, g) to Solver and Observer; in the latter case send withheld to Solver and Observer. Terminate.
- If both Challenger and Solver are corrupt, receive (published, g') or withheld from Challenger and send it to Observer. Terminate.

Note that if Solver is honest, Challenger cannot change the grade g that is published, but the challenger can withhold the grade; however, in this case the functionality announces this to Observer (and a penalty maybe imposed on Challenger for withholding the grade). If Challenger and Solver collude, they are allowed to publish an arbitrary grade to Observer; this corresponds to the idea that the transaction should be seen as between Challenger and Solver, and Observer cannot rely on the grades that are published.

Private Feedback. Though we have formulated only a public grade, the OxSP functionality is flexible enough to implement private feedback as well. This can be done using a puzzle P that, as part of the solution y, accepts a one-time pad from Solver, and uses it to mask a part of the grade.

Definition 2.2 (OxSP: Security). An OxSP scheme (setup, mkPzl, mkSol, evSol, chkGrd) is said to be *secure* if the following 3-party protocol (with a trusted setup, and over a broadcast channel) is a secure realization[2] of the OxSP functionality against active corruption of either party:

- Setup: $\alpha \leftarrow \mathsf{setup}$ is published as the setup.
- Challenger, on input (P, x), computes $(P, Z, \sigma) \leftarrow \mathsf{mkPzl}(\alpha, P, x)$, and broadcasts (P, Z).
- Solver, on input y, and on receiving (P, Z) from Challenger, computes $L \leftarrow \mathsf{mkSol}(\alpha, P, Z, y)$ and broadcasts it.
- Challenger, on receiving L, computes $(v, G) \leftarrow \mathsf{evSol}(\alpha, \sigma, L)$, broadcasts G, and outputs v.
- Observer computes $\mathsf{chkGrd}(\alpha, P, Z, L, G)$; if it accepts she outputs g where G has the form (g, π); else, she outputs $\perp$.

[2]This security definition of OxSP can be instantiated for UC-security or standalone security, by interpreting "secure realization" accordingly. See discussion later.

3 Garbled Circuit Preliminaries

A garbling scheme is defined as a tuple of algorithms $\mathcal{G} = (\mathsf{Gb}, \mathsf{En}, \mathsf{De}, \mathsf{Ev})$, the first of which is randomized. For a security parameter k and a boolean circuit C, $\mathsf{Gb}(1^k, \mathsf{C})$ returns a tuple (F, e, d) or $\perp$, where F is the string representing a garbled version of C, e is an input encoding function and d is an output decoding function. Given an input $\mathbf{x}$, $\mathsf{En}(e, \mathbf{x})$ returns the encoded input $\mathbf{X}$; given the encoded input $\mathbf{X}$, the algorithm $\mathsf{Ev}(F, \mathbf{X})$ returns the encoded output $\mathbf{Y}$; and, given such an encoded output, the algorithm $\mathsf{De}(d, \mathbf{Y})$ returns the raw output $\mathbf{y}$.

Definition 3.1 (Correctness). A garbling scheme (Gb, En, De, Ev) for a circuit family $\mathcal{C}$ is correct if for every circuit $\mathsf{C} \in \mathcal{C}$ and every $\mathbf{x} \in \{0,1\}^{|\mathsf{C}_{\text{in}}|}$, $\Pr_{(F,e,d)\leftarrow \mathsf{Gb}(1^k,\mathsf{C})}[\mathsf{De}(d, \mathsf{Ev}(F, \mathsf{En}(e, \mathbf{x}))) \neq \mathsf{C}(\mathbf{x})]$ is negligible in k.

For the security definition, we need to introduce a leak function $\phi(\cdot)$ that leaks some information about the circuit. The garbling scheme does not hide the leaked information. Typically, this is the topology of the circuit, i.e. the connections between the gates, including the identity of the input and output gates. Given a leakage function ϕ, we define the Sim security for a garbled circuit, which corresponds to the equivalent notions of PrvSim and PrvInd as defined in [7].

*Definition 3.2 (*Sim *Security).* A garbling scheme (Gb, En, De, Ev) for a circuit family $\mathcal{C}$ is said to be Sim secure w.r.t. a leakage function ϕ, if there exists a two-stage PPT simulator $(\mathcal{S}_1, \mathcal{S}_2)$[3] such that, for any two-stage PPT adversary $(\mathcal{A}_1, \mathcal{A}_2)$,[4]

$$\left| \Pr_{\substack{(\sigma,\mathsf{C},\mathbf{x})\leftarrow\mathcal{A}_1 \\ (F,e,d)\leftarrow\mathsf{Gb}(1^k,\mathsf{C}) \\ \mathbf{X}\leftarrow\mathsf{En}(e,\mathbf{x})}} [\mathcal{A}_2(\sigma, F, \mathbf{X}, d) = 1] - \Pr_{\substack{(\sigma,\mathsf{C},\mathbf{x})\leftarrow\mathcal{A}_1 \\ (F,\mathbf{X})\leftarrow\mathcal{S}_1(1^k,\phi(\mathsf{C})) \\ d\leftarrow\mathcal{S}_2(F,\mathsf{Ev}(F,\mathbf{X}),\mathsf{C}(\mathbf{x}))}} [\mathcal{A}_2(\sigma, F, \mathbf{X}, d) = 1] \right|$$

is a negligible function of k.

Another useful property of a GC that we shall require is *authenticity* [7], which makes it infeasible for an adversary given a GC for a circuit C and input labels for one input $\mathbf{x}$ to produce an encoding of an output other than that of $\mathsf{C}(\mathbf{x})$. The formal definition follows.

*Definition 3.3 (*Aut *Security).* A garbling scheme (Gb, En, De, Ev) for a circuit family $\mathcal{C}$ is said to be Sim secure w.r.t. a leakage function ϕ, if for any two-stage PPT adversary $(\mathcal{A}_1, \mathcal{A}_2)$, the following

[3]We allow $\mathcal{S}_2$ to receive the garbled circuit F as input from S_1, to account for common parameters. This could be avoided when no common parameters are involved.

[4]We use the convention that if a PPT adversary does not produce an output of the required form — in this case, $(\sigma, \mathsf{C}, \mathbf{x})$ where $\mathsf{C} \in \mathcal{C}$, $\mathbf{x}$ a valid input for C, and $\sigma \in \{0,1\}^*$ — within its allotted running time, appropriate default values will be used.

probability is negligible in k.

$$\Pr_{\substack{(\sigma,\mathsf{C},\mathbf{x})\leftarrow\mathcal{A}_1\\(F,e,d)\leftarrow\mathsf{Gb}(1^k,\mathsf{C})\\\mathbf{X}\leftarrow\mathsf{En}(e,\mathbf{x}),\mathbf{Y}\leftarrow\mathcal{A}_2(\sigma,F,\mathbf{X})}}[\mathsf{De}(\mathbf{Y})\notin\{\perp,\mathsf{C}(\mathbf{x})]$$

Circuit Notation. Recall that a boolean circuit C can be specified by a directed acyclic graph, with nodes being gates and edges being wires. The source nodes of the DAG (i.e., nodes without any incoming edges) are the input gates, and the sink nodes (those without outgoing edges) are the output gates. Every non-input gate has exactly two input wires,[5] and also has an associated "gate type" (the binary operator implemented by the gate). The *topology* of a circuit C, denoted as $\mathsf{Topo}(\mathsf{C})$ is simply the directed acyclic graph associated with C, which does not include the gate types. For brevity, we will use C to also denote the set of gates in the circuit C, and write $\mathsf{C}_{\text{in}} \subseteq \mathsf{C}$ and $\mathsf{C}_{\text{out}} \subseteq \mathsf{C}$ to denote the set of input gates and output gates respectively.

Given a circuit C each of its wires is identified with the gate from which the wire is output. It will be convenient to denote the two inputs wires feeding into a (non-input) gate $g \in \mathsf{C}$ as $\mathsf{Lft}(g)$ and $\mathsf{Rgt}(g)$; its gate type is denoted as op_g.

4 OxSP from Garbled Circuits

First, we note that since a single puzzle must be available for all the solvers, the puzzle itself cannot be implemented using GCs (which can be evaluated securely only once). Instead, GCs will be used to encode the solutions.

To check if a solution satisfies the puzzle, this GC will need to be evaluated. While evaluating a GC keeps the inputs to the circuit hidden, it also hides the gates of the circuit itself. So to prevent a cheating solver from altering the puzzle, we will need a proof that the GC faithfully implements the puzzle. This is where we rely on our GC construction supporting efficient-to-construct ZK proofs of correct garbling.

To enable publicly checking the grade posted by the challenger, we use the following idea, which is a variation on a common approach used to authenticate purported garbled circuit outputs. The idea is that if an output wire's decoding information is not given to the challenger, but it is able to produce the output label o for a value b for that wire, then that will convince the garbler that the wire indeed evaluated to b.[6] To make it convincing publicly, the garbler publishes a pair (h^0, h^1) where h^b is the hash of the output label for the bit b under a one-way hash function H; now anyone can check that the label published by the challenger o is such that $H(o) = h^b$.

In more detail, the OxSP construction is as follows:

- setup: This generates the setup for a 2-round OT scheme and the setup for a ZK-SNARK scheme that is used below.
- mkPzl: Given input (P, x), where $|x| = \ell$, mkPzl outputs (P, Z, σ) where $Z = \{z_i\}_{i=1}^{\ell}$ and $\sigma = \{\sigma_i\}_{i=1}^{\ell}$ are computed as follows: For each $i = 1$ to ℓ, using the ith bit of x as the choice-bit, the receiver's program in the OT protocol is executed (with the setup for the OT protocol obtained from the setup above), resulting in a message in the OT protocol z_i and a state σ_i.
- mkSol: Given inputs P, Z, y, first, a GC for the circuit P is constructed as $(\tilde{P}, e, d) \leftarrow \mathsf{Gb}(P)$. Then $L = (\tilde{P}, \tilde{Y}, W_Z, d_{\text{view}}, h_{\text{grade}}, \pi)$ is computed as follows:
  - $\tilde{P}$ is the garbled circuit produced by $\mathsf{Gb}(P)$.
  - $\tilde{Y}$ consists of a label for each input wire corresponding to y.
  - $W_Z = \{w_i\}_{i=1}^{\ell}$ where w_i is computed as the response to the message z_i in the OT protocol, given input string (e_i^0, e_i^1) which are the two labels for the wire corresponding to x_i; these strings are specified by e produced by $\mathsf{Gb}(P)$.
  - $d_{\text{view}} = \{d_{v_i}^0, d_{v_i}^1\}_{i=1}^{|v|}$, where each v_i is the index of an output wire *other than the grade output*, and $(d_{v_i}^0, d_{v_i}^1)$ are the two labels for that wire as specified in the decoding information d.
  - $h_{\text{grade}} = \{h_i^0, h_i^1)\}_{i=1}^{|g|}$ where $h_i^b = H(d_{\gamma_i}^b)$, with H being a one-way hash function, and γ_i being the index of the ith grade-output bit.
  - π is a ZK-SNARK proof that all of the above items were generated as prescribed, using the setup that is part of the OxSP scheme's setup described above. Note that the statement proven is specified by (P, Z, L') where L' consists of all the elements of L except the proof π.
- evSol: Given (σ, L), first parse L as $(\tilde{P}, \tilde{Y}, W_Z, d_{\text{view}}, h_{\text{grade}}, \pi)$ as mentioned above. Verify the proof π. If the verification fails, output $(\perp, \perp)$; else, proceed as follows:
  - For $i = 1$ to $|x|$, extract a label for x_i using (w_i, σ_i) in the receiver's program in the OT protocol.
  - Evaluate $\tilde{P}$ using inputs labels for x (extracted above) and y (given as $\tilde{Y}$); use d_{view} to decode v, the view part of the output; the output labels corresponding to the grade wires $(o_1, \ldots, o_{|g|})$ are collected as part of the grade certificate G. G also includes the grade itself, decoded by checking that, for each i, $h_i^{g_i} = H(o_i)$.
  - Output (G, v).
- chkGrd: On input (P, Z, L, G) proceed as follows:
  - If G consists of g and a set of labels $(o_i, \ldots, o_{|g|}$, check that for all i, $h_i^{g_i} = H(o_i)$. If so accept, else reject.
  - If $G = \perp$, verify the proof π included in L. Accept if the verification fails; else reject.

Security Analysis. We present a simulator to prove that the OxSP scheme above satisfies the security definition. For this we shall assume that the ZK-SNARK employed is a UC-secure ZK-Proof of Knowledge,[7] and that the 2-round OT scheme is also UC secure. (If these primitives are only standalone secure, the definition will be satisfied with standalone security.) Also, we model the hash function as a Random Oracle, so that the hash reveals no information at all about the pre-image until it is guessed correctly.[8] The simulation relies on Sim and Aut security of the GC scheme employed.

[5] This is essentially w.l.o.g., as NOT gates can merged into the logic of the gates into which they feed in, or the gate from which it emanates. NOT gates leading to output gates can be handled by the decoding function.

[6] While not required of all "garbling schemes," this property holds for a gate-by-gate garbled circuit [7], which we formalize in the next section.

[7] In fact, it is enough to assume UC-security for the commitment of input y (which can be achieved separately) and assume only soundness of the SNARK. While this can be relevant to reducing the complexity of the SNARK proof systems, for simplicity, the proof below uses UC-secure ZK-PoK.

[8] With the gate-by-gate garbling construction in the next section, it is enough to assume that the hash function is a one-way function w.r.t. the distribution of labels generated by the algorithm sample. But at this point, the definition of Aut by itself does not

If both Challenger and Solver are corrupt, the simulation is easy, since the ideal functionality allows (corrupt) Challenger to specify any output for Observer. If both are honest, then too the security follows from the correctness of the garbled circuit. We focus on the interesting cases where exactly one of Challenger and Solver is corrupt.

Firstly, we note that the ZK-SNARK proof (included as part of L) and its verification can be replaced by an invocation of the ideal ZK-PoK functionality. So we will carry out our analysis in the "ZK-PoK hybrid" model. On the other hand, we shall use the 2-round OT protocol's simulator in a non-blackbox manner, since the ZK-SNARK proof will refer to the messages sent in that protocol.

Corrupt Challenger. If Challenger is corrupt, the simulator will use the OT protocol's simulator to extract x from the Z (OT first message), and combine it with P (sent in the clear) to create the input (P, x) for the OxSP functionality. To simulate the reply L (except the SNARK, which is replaced by an ideal functionality call), the simulator needs to create $\tilde{P}$ (a GC for P), $\tilde{Y}$ (labels for $\tilde{P}$ correspoding to a unknown input y), OT reponses W_Z, and decoding/verifying information d_{view} and h_{grade}. While it could have faithfully generated $\tilde{P}$ (since it knows P), it will not be able to do so for $\tilde{Y}$. So the simulator relies on Sim security of the GC to sample a simulated GC, the input labels for the inputs (x, y) and the decoding information. $\tilde{Y}$ is derived from these input labels. W_Z is simulated using the OT protocol's simulator, corresponding to a corrupt receiver receiving the labels for x. d_{view} and h_{grade} are derived from the simulated decoding information. The simulator also simulates a notification from the ZK-PoK functionality that all the items were correctly constructed. Till this point, the simulation relies only on Sim security, the simulation security for the OT protocol, and the ideal nature of ZK-PoK functionality. Finally, by Aut security, any grade certificate G output by Challenger cannot (except with negligible probability) have a correct output label other than for the bits in g, where $(g, v) = P(x, y)$. Conditioned on authenticity holding, the following completes the simulation: if G has a wrong output label, the simulator sends withhold to the ideal functionality, and otherwise sends $(\mathsf{publish}, g)$.

Corrupt Solver. If Solver is corrupt (and Challenger is honest), the simulator first obtains P from the functionality and sends it to Solver along with simulated OT first messages Z. On receiving L (except the proof component) and the witnesses for proving its correctness (sent to the simulated ZK-PoK) functionality from Solver, the simulator first checks if the witnesses are valid; if not, it sends $\perp$ to the functionality, and terminates. Otherwise, from the valid witness, the simulator extracts y and sends it to the OxSP functionality. This will result in the functionality sending (v, g) to Challenger. In turn, Solver receives $(\mathsf{published}, g)$ from the functionality, and it will simulate a grade certificate G by including the label for each bit of g that it extracted from the witness that Solver had sent to the ZK-PoK functionality.

By comparing with the protocol execution, it can be seen that this (straight-line, blackbox) simulation is a valid simulation in the ZK-PoK hybrid model.

guarantee what effect auxiliary information about labels will have on an adversary's ability to succeed in the Aut experiment.

5 Garbling Gate-by-Gate: A Formalism

In this section, we develop a formalism for garbling a circuit gate-by-gate that will be used to simplify the presentation of our schemes and their security analysis.[9] In particular, we give a security definition for garbling a single gate, and show that if an entire circuit is garbled gate-by-gate using such a secure gate-garbling scheme, then the resulting garbled circuit will also be secure according to the standard security notion for garbling schemes, given in Definition 3.2

The garbling scheme will be fully specified given a function gateGb that garbles a single gate. The function gateGb takes the type of a gate, along with labels for its input and output wires, and produces the description of the corresponding garbled gate. To garble the entire circuit, we shall first sample labels for all the wires, and then (in parallel) call gateGb on all the gates of the circuit with the appropriate labels.

Given gateGb, along with two auxiliary probabilistic algorithms – setup for creating common parameters[10] and sample sampling labels from a (security-parameter dependent) label space $\mathbb{F}$– we define the five tuple of algorithms (Gb, En, De, Ev) as follows. Below k denotes the security parameter.

- $\mathsf{Gb}(1^k, \mathsf{C})$:
  (1) Generate $\mathsf{par} \leftarrow \mathsf{setup}(1^k, \mathsf{Topo}(\mathsf{C}))$
  (2) For every gate $g \in \mathsf{C}$, choose $\mathcal{K}_g^0 \leftarrow \mathsf{sample}(\mathsf{par})$ and $\mathcal{K}_g^1 \leftarrow \mathsf{sample}(\mathsf{par})$.
  (3) For every $g \in \mathsf{C} \setminus \mathsf{C}_{\text{in}}$, compute
  $$\mathbf{\Psi}_g \leftarrow \mathsf{gateGb}(\mathsf{op}_g, \mathcal{K}^0_{\mathsf{Lft}(g)}, \mathcal{K}^1_{\mathsf{Lft}(g)}, \mathcal{K}^0_{\mathsf{Rgt}(g)}, \mathcal{K}^1_{\mathsf{Rgt}(g)}, \mathcal{K}^0_g, \mathcal{K}^1_g, g, \mathsf{par})$$
  (4) Set $F = (\{\mathbf{\Psi}_g\}_{g \in \mathsf{C} \setminus \mathsf{C}_{\text{in}}}, \mathsf{par})$.
  (5) Define e such that for each $g \in \mathsf{C}_{\text{in}}$, $e(g, 0) = \mathcal{K}_g^0$ and $e(g, 1) = \mathcal{K}_g^1$.
  (6) Define d such that for each $g \in \mathsf{C}_{\text{out}}$, $d(g, \mathcal{K}_g^0) = 0$ and $d(g, \mathcal{K}_g^1) = 1$.
  (7) Return (F, e, d)
- $\mathsf{En}(e, \mathbf{x})$: For every input gate $g \in \mathsf{C}_{\text{in}}$, set $\mathbf{X}_g = e(g, \mathbf{x}_g)$, where $\mathbf{x} = \{\mathbf{x}_g\}_{g \in \mathsf{C}_{\text{in}}}$. Return $\mathbf{X} := \{\mathbf{X}_g\}_{g \in \mathsf{C}_{\text{in}}}$.
- $\mathsf{De}(d, \mathbf{Y})$: For every output gate $g \in \mathsf{C}_{\text{out}}$, set $\mathbf{y}_g = d(g, \mathbf{Y}_g)$, where $\mathbf{Y} = \{\mathbf{Y}_g\}_{g \in \mathsf{C}_{\text{out}}}$. Return $\mathbf{y} := \{\mathbf{y}_g\}_{g \in \mathsf{C}_{\text{out}}}$.
- $\mathsf{Ev}(F, \mathbf{X})$: For each $g \in \mathsf{C}_{\text{in}}$, set $W_g = \mathbf{X}_g$. Then, for every $g \in \mathsf{C} \setminus \mathsf{C}_{\text{in}}$, in topologically sorted order so that g occurs after $\mathsf{Lft}(g)$ and $\mathsf{Rgt}(g)$, set $W_g := \mathbf{\Psi}_g(W_{\mathsf{Lft}(g)}, W_{\mathsf{Rgt}(g)}, \mathsf{par})$, where $F = (\{\mathbf{\Psi}_g\}_{g \in \mathsf{C} \setminus \mathsf{C}_{\text{in}}}, \mathsf{par})$. Finally, return the vector $\mathbf{Y} := \{W_g\}_{g \in \mathsf{C}_{\text{out}}}$.

We define a security property for gateGb that would lead to the security of the full garbled circuit. In $\mathsf{Exp}^{\mathsf{gateGb}}_{\mathcal{C}, \mathcal{A}^*, \mathcal{S}^*}(b)$ (Figure 2) we allow a three-stage adversary to first provide not only a circuit C, but also all the labels for all the wires in the circuit, *except* one label each for the two input wires of a single gate g^*. These two "missing labels" will be sampled randomly. Then, the security requirement is that the adversary cannot distinguish between a garbled circuit obtained by gate-by-gate garbling of C using all these labels, and

[9] Compared to a prior formalism in the literature [8], which works layer-by-layer, our gate-by-gate formalism is easier to use and does not incur costs due to transforming the circuit into a layered circuit.

[10] Common parameters are allowed for generality, but also it can be useful for improving efficiency, as we shall see later.

$\mathsf{Exp}^{\mathsf{gateGb}}_{\mathcal{C},\mathcal{A}^*,\mathcal{S}^*}(b)$, where gateGb is a gate-garbling algorithm with associated algorithms setup and sample, $\mathcal{C}$ is a circuit family, $\mathcal{A}^* = (\mathcal{A}^*_1, \mathcal{A}^*_2, \mathcal{A}^*_3)$ is a three-stage adversary, $\mathcal{S}^*$ a simulator, and $b \in \{0,1\}$. k denotes the security parameter.

(1) $(\sigma^*_1, \mathsf{C}) \leftarrow \mathcal{A}^*_1(1^k)$ where σ^*_1 is a state (to be passed on to $\mathcal{A}^*_2$), and $\mathsf{C} \in \mathcal{C}$.

(2) $\mathsf{par} \leftarrow \mathsf{setup}(1^k, \mathsf{Topo}(\mathsf{C}))$, and for each $g \in \mathsf{C} \setminus \mathsf{C}_{\mathsf{in}}$ and $x \in \{0,1\}$, $\mathcal{K}^x_g \leftarrow \mathsf{sample}(\mathsf{par})$.

(3) $(\sigma^*_2, g^*, a^*, b^*) \leftarrow \mathcal{A}^*_2(\sigma^*_1, \mathsf{par})$ where σ^*_2 is a state, $g^* \in \mathsf{C} \setminus \mathsf{C}_{\mathsf{in}}$ s.t. $\mathsf{Lft}(g^*), \mathsf{Rgt}(g^*) \in \mathsf{C}_{\mathsf{in}}$, $a^*, b^* \in \{0,1\}$.

(4) For each gate $g \in \mathsf{C} \setminus \mathsf{C}_{\mathsf{in}}$,

$\mathbf{\Psi}_g \leftarrow \mathsf{gateGb}(\mathsf{op}_g, \mathcal{K}^0_{\mathsf{Lft}(g)}, \mathcal{K}^1_{\mathsf{Lft}(g)}, \mathcal{K}^0_{\mathsf{Rgt}(g)}, \mathcal{K}^1_{\mathsf{Rgt}(g)}, \mathcal{K}^0_g, \mathcal{K}^1_g, g, \mathsf{par})$

(5) If $b = 1$, replace $\mathbf{\Psi}_{g^*}$ as follows:

$$\mathbf{\Psi}_{g^*} \leftarrow \mathcal{S}^*(\mathcal{K}^{a^*}_{\mathsf{Lft}(g^*)}, \mathcal{K}^{b^*}_{\mathsf{Rgt}(g^*)}, \mathcal{K}^{\mathsf{op}_{g^*}(a^*,b^*)}_{g^*}, g, \mathsf{par}).$$

(6) Return the bit

$b' \leftarrow \mathcal{A}^*_3\left(\sigma^*_2, \{\mathbf{\Psi}_g\}_{g \in \mathsf{C}\setminus\mathsf{C}_{\mathsf{in}}}, \{\mathcal{K}^x_g\}_{(g,x) \notin \{(\mathsf{Lft}(g^*), 1-a^*), (\mathsf{Rgt}(g^*), 1-b^*)\}}\right)$

Figure 2: Experiment $\mathsf{Exp}^{\mathsf{gateGb}}_{\mathcal{C},\mathcal{A}^*,\mathcal{S}^*}(b)$ for defining GateSim security of gateGb

a simulated garbled circuit in which only the gate g^* is simulated. The simulation should use only the input labels provided by the adversary and the relevant output label, and should not use the gate type nor the other labels of the circuit.

Definition 5.1 (GateSim Security). A gate-garbling algorithm gateGb is said to be GateSim secure for a circuit family $\mathcal{C}$ if there exists a PPT simulator $\mathcal{S}^*$ such that for any PPT three-stage adversary $\mathcal{A}^*$,

$$\mathsf{Adv}^{\mathsf{gateGb}}_{\mathcal{C},\mathcal{A}^*,\mathcal{S}^*} := \left|\Pr[\mathsf{Exp}^{\mathsf{gateGb}}_{\mathcal{C},\mathcal{A}^*,\mathcal{S}^*}(0) = 1] - \Pr[\mathsf{Exp}^{\mathsf{gateGb}}_{\mathcal{C},\mathcal{A}^*,\mathcal{S}^*}(1) = 1]\right|$$

is negligible in the security parameter.

In Appendix A we prove the following theorem.

Theorem 5.2. *A gate-by-gate garbling scheme* (Gb, En, De, Ev) *that uses a gate garbling scheme* gateGb *that is* GateSim *secure for a circuit-family $\mathcal{C}$, is* Sim *secure for $\mathcal{C}$ w.r.t. the circuit topology as the leakage.*

We also a define a property for a gate-garbling algorithm, called GateAut that, in combination with Sim security, ensures Aut security. This property is designed following the approach in [7] for proving Aut security of a typical gate-by-gate garbling protocol. Informally, the GateAut property states that if a constant gate is garbled, then the garbled gate function is statistically independent of the output label that is never output.

Definition 5.3. A gate-garbling algorithm gateGb is said to be GateAut secure if there exist functions gateGb_0 and gateGb_1 such that for any gate g such that op_g is the constant function which outputs the bit b,

$$\begin{aligned}&\mathsf{gateGb}(\mathsf{op}_g, \mathcal{K}^0_{\mathsf{Lft}(g)}, \mathcal{K}^1_{\mathsf{Lft}(g)}, \mathcal{K}^0_{\mathsf{Rgt}(g)}, \mathcal{K}^1_{\mathsf{Rgt}(g)}, \mathcal{K}^0_g, \mathcal{K}^1_g, g, \mathsf{par})\\ &\quad = \mathsf{gateGb}_b(\mathcal{K}^0_{\mathsf{Lft}(g)}, \mathcal{K}^1_{\mathsf{Lft}(g)}, \mathcal{K}^0_{\mathsf{Rgt}(g)}, \mathcal{K}^1_{\mathsf{Rgt}(g)}, \mathcal{K}^b_g, g, \mathsf{par})\end{aligned}$$

for all inputs of gateGb.

Now we state our result for authenticated garbling, which is similar to a result in [7] (Theorem 13 in the full-version).

Theorem 5.4. *If a gate-by-gate garbling scheme* (Gb, En, De, Ev) *using a gate garbling scheme* gateGb *is* Sim *secure w.r.t. the circuit topology as the leakage for a circuit-family $\mathcal{C}$ which allows constant output gates, then it is also* Aut *secure provided* gateGb *is* GateAut *secure and the distribution of labels generated by* sample *has min-entropy $\omega(\log k)$.*

Proof. Given a circuit C and input $\mathbf{x}$, consider a circuit C′ identical to C in its topology, but with all output gates replaced by constant gates producing the same bit as in the output of $\mathsf{C}(\mathbf{x})$. By Sim security, one cannot distinguish between $(F, \mathbf{X}, d)$ generated by garbling C and encoding $\mathbf{x}$, and the same generated by garbling C′ instead. So it is enough to bound the adversary's success probability when the latter is used in the Aut experiment (Definition 3.3).

In the modified experiment using C′, we appeal to GateAut security, to note that the adversay's view is independent of the labels of the output gates corresponding to the bits in the complement of $\mathsf{C}(\mathbf{x})$.

Now, an adversary can produce an encoded output that decodes to a value other than $\mathsf{C}(\mathbf{x})$ only if it can guess the label for at least one bit in the complement. The min-entropy condition of sample ensures that the adversary has negligible probability in guessing any label that is independent of its view. □

6 ZK-Friendly Garbling from DDH

Before we describe our construction, we fix some notational conventions for elliptic curve groups.

Notation. Recall that an order p elliptic curve group $\mathbb{G}$ over a prime field $\mathbb{F}_q$ is a set of p points in $(\mathbb{F}_q \times \mathbb{F}_q) \cup \{\mathbf{O}\}$ with a certain group operation defined over them ($\mathbf{O}$ denoting the identity element); we will require the group's order p to be a prime. We will use the multiplicative notation for $\mathbb{G}$. In particular, given $\mathbf{g} \in \mathbb{G}$ and $\ell \in \mathbb{F}_p$, we write $\mathbf{g}^\ell$ to combine ℓ instances of g using the group operation (treating ℓ as an integer); it will be convenient to extend this notation to $\ell \in \mathbb{F}_q$ as well. Also, $\mathbf{O} = \mathbf{g}^0$. We shall rely only on the DDH assumption over this group for the security of the garbled circuit. We will use elements in $\mathbb{F}_q$ as the labels (though they will be used modulo p as well), and also rely on a simple function $\mathsf{x} : \mathbb{G} \setminus \{\mathbf{O}\} \to \mathbb{F}_q$, which simply maps a point in $\mathbb{F}_q \times \mathbb{F}_q$ to its x-coordinate.

Below, the parameters include a set of randomly sampled and fixed generator points $(\mathbf{g}_1, \mathbf{h}_1)$, $(\mathbf{g}_2, \mathbf{h}_2)$, ..., $(\mathbf{g}_{2\mu-1}, \mathbf{h}_{2\mu-1})$. The reason for restricting to such a set (rather than using a different pair for each gate) is that this set of points can support any circuit that has a fan-out of at most μ. Since a wire has a maximum fan-out of μ, a gate with two input wires can have at most $2(\mu-1)$ neighbors (i.e., gates that share at least one input wire). Since there are $2\mu - 1$ $(\mathbf{g}_i, \mathbf{h}_i)$ pairs available, a gate can always be assigned a pair that is not assigned to any of its neighboring gates.

Algorithm EC-gateGb**:**

- $\mathsf{gateGb}(\mathsf{op}_g, \mathcal{K}^0_{\mathsf{Lft}(g)}, \mathcal{K}^1_{\mathsf{Lft}(g)}, \mathcal{K}^0_{\mathsf{Rgt}(g)}, \mathcal{K}^1_{\mathsf{Rgt}(g)}, \mathcal{K}^0_g, \mathcal{K}^1_g, g, \mathsf{par})$ where $\mathsf{par} = \{\mathsf{Topo}(\mathsf{C}), (\mathbf{g}_1, \mathbf{h}_1), \ldots, (\mathbf{g}_{2\mu-1}, \mathbf{h}_{2\mu-1})\}$.

(1) Use par to choose $(\mathbf{g}, \mathbf{h})$ from among $\{\mathbf{g}_i, \mathbf{h}_i\}_{i=1}^{2\mu-1}$ such that the same pair is not chosen for any gate which shares an input wire with g.

(2) For $b \in \{0,1\}$, let

$$r_b = \mathrm{x}(\mathbf{g}^{\mathcal{K}^b_{\mathsf{Lft}(g)}}), \qquad s_b = \mathrm{x}(\mathbf{h}^{\mathcal{K}^b_{\mathsf{Lft}(g)}}),$$
$$t_b = \mathrm{x}(\mathbf{g}^{\mathcal{K}^b_{\mathsf{Rgt}(g)}}), \qquad u_b = \mathrm{x}(\mathbf{h}^{\mathcal{K}^b_{\mathsf{Rgt}(g)}}).$$

(3) Let $\mathbf{M} = [A\ B\ C] \in \mathbb{F}_q^{4\times 12}$ defined as follows. The four rows of A, B, C are indexed by $(a,b) \in \{0,1\}^2$, and are defined as:

$$A_{a,b} = \begin{bmatrix} r_a & r_a^2 & s_a & s_a^2 \end{bmatrix} \qquad B_{a,b} = \begin{bmatrix} t_b & t_b^2 & u_b & u_b^2 \end{bmatrix}$$

and $C_{a,b} = A_{a,b} \odot B_{a,b}$ where $\odot$ is the element-wise product.

(4) If any entry in $\mathbf{M}$ is 0, or if $\mathbf{M}$ is not full-rank, return $\perp$.

(5) For $a, b \in \{0,1\}$, let

$$\alpha_{a,b} = \mathcal{K}_g^{\mathsf{op}_g(a,b)}$$

(6) Jointly sample $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{F}_q^4$ uniformly conditioned on

$$\begin{bmatrix} \alpha_{0,0} \\ \alpha_{0,1} \\ \alpha_{1,0} \\ \alpha_{1,1} \end{bmatrix} = \mathbf{M} \begin{bmatrix} \mathbf{a} \\ \mathbf{b} \\ \mathbf{c} \end{bmatrix}$$

(7) Define the polynomial $P(X_1, X_2, Y_1, Y_2) = \langle (X_1, X_1^2, X_2, X_2^2), \mathbf{a} \rangle + \langle (Y_1, Y_1^2, Y_2, Y_2^2), \mathbf{b} \rangle + \langle (X_1Y_1, X_1^2Y_1^2, X_2Y_2, X_2^2Y_2^2), \mathbf{c} \rangle$. Return the function $\mathbf{\Psi}$ (described by the vector $[\mathbf{a}\ \mathbf{b}\ \mathbf{c}] \in F_q^{12}$), defined as

$$\mathbf{\Psi}(\mathcal{K}_L, \mathcal{K}_R) = P\left(\mathrm{x}(\mathbf{g}^{\mathcal{K}_L}), \mathrm{x}(\mathbf{h}^{\mathcal{K}_L}), \mathrm{x}(\mathbf{g}^{\mathcal{K}_R}), \mathrm{x}(\mathbf{h}^{\mathcal{K}_R})\right).$$

Correctness and Security Analysis.

Theorem 6.1. *The gate-by-gate garbling algorithm using* EC-gateGb *is correct.*

Proof. It is easy to see that if EC-gateGb does not output $\perp$, then the matrix equation ensures that for every input, it outputs the correct output label.

In case there is a rational point on the elliptic curve such that the X-coordinate is zero, there are at most two such points. The total number of points on the curve is at least $q - 2/\sqrt{q} + 1$. Hence, the maximum probability that a randomly chosen point has an X-coordinate that is zero is at most $2/(q - 2/\sqrt{q} + 1)$. The probability of any of $r_1, s_1, t_1, u_1, r_0, s_0, t_0, u_0$ being zero is at most $16/(q - 2/\sqrt{q} + 1) \leq 32/q$.

If $\mathbf{M}$ is not full-rank, then $\mathbf{M}''$ as defined in lemma C.8 is also not full rank. The maximum probability of that is $52/(2/(q - 2/\sqrt{q} + 1) \leq 104/q$.

Hence, the maximum probability that the garbling algorithm using EC-gateGb does not output a correct garbling is $136/q$. □

To prove that a gate-by-gate garbling scheme is Sim and Aut secure, we need to only prove that EC-gateGb is GateSim and GateAut secure (because of Theorems 5.2 and 5.4. We do this below. We start with the easier part:

Lemma 6.2. EC-gateGb *satisfies* GateAut *security*

Proof. The function $\mathsf{gateGb}_b(\mathcal{K}^0_{\mathsf{Lft}(g)}, \mathcal{K}^1_{\mathsf{Lft}(g)}, \mathcal{K}^0_{\mathsf{Rgt}(g)}, \mathcal{K}^1_{\mathsf{Rgt}(g)}, \mathcal{K}^b_g, g, \mathsf{par})$ is defined as $\mathsf{EC\text{-}gateGb}(\mathsf{op}_g, \mathcal{K}^0_{\mathsf{Lft}(g)}, \mathcal{K}^1_{\mathsf{Lft}(g)}, \mathcal{K}^0_{\mathsf{Rgt}(g)}, \mathcal{K}^1_{\mathsf{Rgt}(g)}, \mathcal{K}^b_g, \mathcal{K}^b_g, g, \mathsf{par})$ for any constant op_g. For a constant op_g, the other output label is never used in the matrix equation in EC-gateGb, and hence they both output the same gate. □

Next, we shall prove GateSim security of EC-gateGb assuming that the DDH assumption holds for the underlying elliptic curve group.[11] Note that theorem relies on the DDH assumption, as well as a requirement that the cofactor of $\mathbb{G}$ is polynomially bounded. As discussed below, the cofactor approximately equals the ratio q/p where $\mathbb{F}_q$ is the base-field and $\mathbb{F}_p$ is the scalar field. It is indeed the case for all typical elliptic curves used in cryptography that the cofactor is small.

Theorem 6.3. *The algorithm* EC-gateGb *over a prime-order elliptic curve group $\mathbb{G}$ where the DDH assumption holds, and for which the cofactor is polynomially bounded, satisfies* GateSim *security.*

Before we prove Theorem 6.3, we summarize two main arguments we shall employ. These two results appear in Lemma 6.5 and Lemma 6.6 below with brief proof sketches (with the somewhat tediously detailed full proofs in Appendix C.1).

(1) While we shall eventually rely on the DDH assumption for the elliptic curve group $\mathbb{G}$, our use of labels as exponents does not immediately correspond to the distribution of exponents in the DDH assumption. The essential difference is that we draw our exponents from the base-field $\mathbb{F}_q$, whereas the standard DDH assumption draws them from the scalar field $\mathbb{F}_p$ (p being the order of $\mathbb{G}$). To analyze this difference, we formulate a variant of the DDH assumption, which we call altDDH, where one of the exponents is drawn from $\mathbb{F}_q$. We show that hardness of altDDH follows tightly from the hardness of DDH.
In the proof of Theorem 6.3, the distributions involved are such that the secret exponents are used to exponentiate many bases. The requisite hardness assumption is captured in the definition of altDDH* (see Definition 6.4). A routine reduction shows that hardness of altDDH implies that of altDDH* as well. Lemma 6.5 presents this result.

(2) The second part of the proof is specific to the details of how EC-gateGb samples the coefficients of the polynomial used to define $\mathbf{\Psi}_g$ for a gate g. Recall that $\mathbf{\Psi}_g$ is a function that maps two input labels into an output label. While the actual garbling picks the coefficients defining $\mathbf{\Psi}_g$ so that it works correctly for all 4 possible combinations of the input labels, a simulation will have to pick it given only one combination of input labels and output label.
The polynomial underlying $\mathbf{\Psi}_g$ is chosen such that we can argue that these two distributions are statistically indistinguishable. The required result is captured in Lemma 6.6. (It is the invocation of this lemma in the proof that introduces the requirement of a bounded cofactor.)

[11] For simplicity, we state the theorem in asymptotic terms. However, our proof presents a concrete analysis.

Definition 6.4. For an elliptic curve group $\mathbb{G}$ over the base-field $\mathbb{F}_q$, and an adversary $\mathcal{A}$, its advantage in the altDDH* *problem* over $\mathbb{G}$, $\mathrm{Adv}^{\mathrm{altDDH}^*}_{\mathbb{G},\mathcal{A}}$, is defined as its advantage in distinguishing between the following two distributions:

$$\left\{\{\mathbf{g}_i,\mathbf{h}_i\}_{i\in[n]},\{\mathbf{g}_i^x,\mathbf{h}_i^x,\mathbf{g}_i^y,\mathbf{h}_i^y\}_{i\in[n]}\right\}_{\substack{x,y\leftarrow\mathbb{F}_q,\\ \forall i\in[n]\ \mathbf{g}_i,\mathbf{h}_i\leftarrow\mathbb{G}}} \quad \textit{and}$$

$$\left\{\{\mathbf{g}_i,\mathbf{h}_i\}_{i\in[n]},\{\mathbf{g}_i^x,\mathbf{h}_i^x,\mathbf{g}_i^y,\mathbf{h}_i^y\}_{i\in[n-1]},\mathbf{g}',\mathbf{h}',\mathbf{g}'',\mathbf{h}''\right\}_{\substack{x,y\leftarrow\mathbb{F}_q,\\ \forall i\in[n]\ \mathbf{g}_i,\mathbf{h}_i\leftarrow\mathbb{G},\\ \mathbf{g}',\mathbf{h}',\mathbf{g}'',\mathbf{h}''\leftarrow\mathbb{G}}}.$$

Lemma 6.5. *For any PPT adversary $\mathcal{A}$ for the* altDDH* *problem on an elliptic curve group $\mathbb{G}$ over the base-field $\mathbb{F}_q$, there is a PPT adversary $\mathcal{A}'$ for the DDH problem on $\mathbb{G}$ such that* $\mathrm{Adv}^{DDH}_{\mathbb{G},\mathcal{A}'} \geq \mathrm{Adv}^{\mathrm{altDDH}^*}_{\mathbb{G},\mathcal{A}}/4 - 16/\sqrt{q}$.

Proof sketch. *(See Appendix C.1 for the full proof.)* First, we show that hardness of the altDDH problem over $\mathbb{G}$ (which is similar to the DDH problem, but one of the exponents is drawn from $\mathbb{F}_q$ instead of $\mathbb{F}_p$) follows tightly from the hardness of DDH over $\mathbb{G}$. This relies on Hasse's Theorem, a classical result on elliptic curves, which shows that the maximum cardinality of an elliptic curve over a base-field $\mathbb{F}_q$ is $N = q \pm O(\sqrt{q})$, and a prime-order subgroup $\mathbb{G}$ of such a curve has order $p = N/h$ for an integer h (called the cofactor of $\mathbb{G}$). This means $q/p = qh/N = h(1 \pm 1/\sqrt{q})$ and hence is very close to an integer. Thus drawing an exponent uniformly from $\mathbb{F}_q$ has the effect of drawing it almost uniformly from $\mathbb{F}_p$ (since exponents work modulo p).

Then we show that hardness of altDDH implies the hardness of altDDH* via a routine hybrid argument. □

Lemma 6.6. *Fix any $a^*, b^* \in \{0,1\}$, $r_{a^*}, s_{a^*}, t_{b^*}, u_{b^*} \in \mathbb{F}_q \setminus \{0\}$, and $\alpha_{0,0}, \alpha_{0,1}, \alpha_{1,0}, \alpha_{1,1} \in \mathbb{F}_q$, and any non-empty subset $I \subseteq \mathbb{F}_q \setminus \{0\}$ such that $|I| \geq \beta q$, for $0 < \beta < 1$. Define a distribution over matrices in $\mathbb{F}_q^{4\times 12}$ as follows:*

(1) $r_{1-a^*}, s_{1-a^*}, t_{1-b^*}, u_{1-b^*} \leftarrow I$ uniformly.

(2) Let $\mathbf{M} = [A\ B\ C] \in \mathbb{F}_q^{4\times 12}$ where rows of A, B, C are indexed by $(a,b) \in \{0,1\}^2$, and are defined as:

$$A_{a,b} = \begin{bmatrix} r_a & r_a^2 & s_a & s_a^2 \end{bmatrix} \quad B_{a,b} = \begin{bmatrix} t_b & t_b^2 & u_b & u_b^2 \end{bmatrix}$$

and $C_{a,b} = A_{a,b} \odot B_{a,b}$.

(3) If $\mathbf{M}$ is not of rank 4, go to step (1). Else output $\mathbf{M}$.

Define the following two distributions over $\mathbb{F}_q^{12}$:

- *Distribution $\mathcal{D}$: Sample $\mathbf{M}$ as above, and output $[\mathbf{a}\ \mathbf{b}\ \mathbf{c}]$ sampled uniformly conditioned on*

$$\begin{bmatrix} \alpha_{0,0} \\ \alpha_{0,1} \\ \alpha_{1,0} \\ \alpha_{1,1} \end{bmatrix} = \mathbf{M} \begin{bmatrix} \mathbf{a} \\ \mathbf{b} \\ \mathbf{c} \end{bmatrix}$$

- *Distribution $\mathcal{U}$: Sample $\mathbf{M}$ as above and output $[\mathbf{a}\ \mathbf{b}\ \mathbf{c}]$ sampled uniformly conditioned on*

$$\alpha_{a^*,b^*} = \mathbf{M}_{a^*,b^*} \begin{bmatrix} \mathbf{a} \\ \mathbf{b} \\ \mathbf{c} \end{bmatrix}$$

Then the statistical difference between $\mathcal{D}$ and $\mathcal{U}$ is at most $\frac{9}{\beta^2\sqrt{q}}$.

Proof. *Sketch [see appendix C.2] for a formal proof* We depend on the well known result that if the probability of collision for a distribution $\mathcal{D}$ over a set $\mathcal{S}$ is $Col(\mathcal{D})$, i.e. $Col(\mathcal{D}) = \Pr_{x,y\leftarrow\mathcal{D}}[x = y]$, then:

$$\sum_{y\in\mathcal{S}} \left| \Pr_{x\leftarrow\mathcal{D}}[x = y] - \Pr_{x\leftarrow\mathcal{U}}[x = y] \right| \leq \sqrt{|\mathcal{S}|Col(\mathcal{D}) - 1}$$

where $\mathcal{U}$ is the uniform distribution over $\mathcal{S}$. In our case, we show that $Col(\mathcal{D}) \leq (1/q^{11} + (58/\beta^2 + 240/\beta^4)/q^{12})$ and $|\mathcal{S}| = q^{11}$, thus giving us the bound. □

Finally, we are ready to prove Theorem 6.3.

Proof of Theorem 6.3. We start by describing the simulator $\mathcal{S}^*$ from the definition of GateSim. The simulated garbled gate $\mathcal{S}^*(\widetilde{\mathcal{K}}_{\mathsf{Lft}(g)}, \widetilde{\mathcal{K}}_{\mathsf{Rgt}(g)}, \widetilde{\mathcal{K}}_g, g, \mathsf{par})$ is sampled as follows:

- Let $r = \mathrm{x}(\mathbf{g}^{\widetilde{\mathcal{K}}_{\mathsf{Lft}(g)}}), s = \mathrm{x}(\mathbf{h}^{\widetilde{\mathcal{K}}_{\mathsf{Lft}(g)}}), t = \mathrm{x}(\mathbf{g}^{\widetilde{\mathcal{K}}_{\mathsf{Rgt}(g)}}), u = \mathrm{x}(\mathbf{h}^{\widetilde{\mathcal{K}}_{\mathsf{Rgt}(g)}})$
- Uniformly sample $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{F}_q^4$ such that

$$\begin{bmatrix} r & r^2 & s & s^2 & t & t^2 & u & u^2 & rt & r^2t^2 & su & s^2u^2 \end{bmatrix} \begin{bmatrix} \mathbf{a} \\ \mathbf{b} \\ \mathbf{c} \end{bmatrix} = \widetilde{\mathcal{K}}_g$$

- Return simulated $\mathbf{\Psi}_g$ described by $[\mathbf{a}\ \mathbf{b}\ \mathbf{c}]$.

Given $\mathcal{A}^* = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$, we need to bound

$$\mathrm{Adv}^{\mathrm{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^*} := \left| \Pr[\mathrm{Exp}^{\mathrm{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^*}(0) = 1] - \Pr[\mathrm{Exp}^{\mathrm{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^*}(1) = 1] \right| \tag{1}$$

Towards this we shall define an intermediate experiment $\mathrm{Exp}^{*\mathrm{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^{**}}$ (using an intermediate simulator $\mathcal{S}^{**}$, defined below). This experiment is identical to $\mathrm{Exp}^{\mathrm{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^*}(1)$, except that Step (5) in Figure 2 is replaced by the following:

$$\mathbf{\Psi}_{g^*} \leftarrow \mathcal{S}^{**}(a^*, b^*, \mathsf{op}_{g^*}, \mathcal{K}^{a^*}_{\mathsf{Lft}(g^*)}, \mathcal{K}^{b^*}_{\mathsf{Rgt}(g^*)}, \mathcal{K}^0_{g^*}, \mathcal{K}^1_{g^*}, g^*, \mathsf{par})$$

where the process on the right hand side is defined as follows:

- Let $r_{a^*} = \mathrm{x}(\mathbf{g}^{\mathcal{K}^{a^*}_{\mathsf{Lft}(g)}}), s_{a^*} = \mathrm{x}(\mathbf{h}^{\mathcal{K}^{a^*}_{\mathsf{Lft}(g)}}), t_{b^*} = \mathrm{x}(\mathbf{g}^{\mathcal{K}^{b^*}_{\mathsf{Rgt}(g)}}), u_{b^*} = \mathrm{x}(\mathbf{h}^{\mathcal{K}^{b^*}_{\mathsf{Rgt}(g)}})$.
- Uniformly sample $\mathbf{g}', \mathbf{h}', \mathbf{g}'', \mathbf{h}'' \leftarrow \mathbb{G} \setminus \{O\}$, conditioned on their y-coordinates not being 0,[12] and let $r_{1-a^*} = \mathrm{x}(\mathbf{g}'), s_{1-a^*} = \mathrm{x}(\mathbf{h}'), t_{1-b^*} = \mathrm{x}(\mathbf{g}''), u_{1-b^*} = \mathrm{x}(\mathbf{h}'')$.
- Let $\mathbf{M} = [A\ B\ C] \in \mathbb{F}_q^{4\times 12}$ where rows of A, B, C are indexed by $(a,b) \in \{0,1\}^2$, and are defined as:

$$A_{a,b} = \begin{bmatrix} r_a & r_a^2 & s_a & s_a^2 \end{bmatrix} \quad B_{a,b} = \begin{bmatrix} t_b & t_b^2 & u_b & u_b^2 \end{bmatrix}$$

and $C_{a,b} = A_{a,b} \odot B_{a,b}$.

- For all $a, b \in \{0,1\}$, let $\alpha_{a,b} = \mathcal{K}^{\mathsf{op}_{g^*}(a,b)}_{g^*}$.

[12] This conditioning is to ensure that their x-coordinates are chosen uniformly from a set $I \subseteq \mathbb{F}_q$, which will allow the convenience of applying Lemma 6.6. The uniformity follows because for each such group element there is exactly one another element with the same x-coordinate.

- Uniformly sample $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{F}_q^4$ such that

$$\begin{bmatrix} \alpha_{0,0} \\ \alpha_{0,1} \\ \alpha_{1,0} \\ \alpha_{1,1} \end{bmatrix} = \mathbf{M} \begin{bmatrix} \mathbf{a} \\ \mathbf{b} \\ \mathbf{c} \end{bmatrix}$$

- Return simulated $\mathbf{\Psi}_{g^*}$ described by $[\mathbf{a}\ \mathbf{b}\ \mathbf{c}]$.

Now,

$$\begin{aligned} \mathrm{Adv}^{\mathrm{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^*} \leq & \left| \Pr[\mathrm{Exp}^{\mathrm{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^*}(1) = 1] - \Pr[\mathrm{Exp}^{*\mathrm{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^{**}} = 1] \right| \\ & + \left| \Pr[\mathrm{Exp}^{*\mathrm{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^{**}} = 1] - \Pr[\mathrm{Exp}^{\mathrm{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^*}(0) = 1] \right| \end{aligned} \tag{2}$$

For the first component, the difference between the two experiments – namely simulation using $\mathcal{S}^*$ and using $\mathcal{S}^{**}$ – corresponds to sampling $[\mathbf{a}\ \mathbf{b}\ \mathbf{c}]$ according to the distribution $\mathcal{U}$ or $\mathcal{D}$ in Lemma 6.6, with the set I being the set of non-zero x-coordinates of the points in the elliptic curve group $\mathbb{G}$. Hence, this component is at most $\frac{9}{\beta^2\sqrt{q}}$, for $\beta \leq |I|/q$. We note that by the requirement that $p/q \geq \eta$, we can take $\beta = \eta/2 - 2/q \leq (p-3)/2q \leq |I|/q$; the last inequality follows from the fact that there are at most 3 points in $\mathbb{G}$ that are omitted by the conditioning used by $\mathcal{S}^{**}$ to sample each of $\mathbf{g}', \mathbf{h}', \mathbf{g}'', \mathbf{h}''$ (namely, the identity O and at most two points with y-coordinate 0), and among the other points, each point has exactly one point with the same x-coordinate.

Turning to the second component, if $\mathcal{A}^*$ results in a non-negligible distinguishing between $\mathrm{Exp}^{*\mathrm{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^{**}}$ and $\mathrm{Exp}^{\mathrm{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^*}(0)$, we shall use it to build an adversary $\mathcal{A}$ for altDDH* in Lemma 6.5 with a non-negligible advantage. For this, note that the only difference between the two experiments is in how $r_{1-a^*}, s_{1-a^*}, t_{1-b^*}, u_{1-b^*}$ are defined:

- In one case, they are derived from the four group elements $\mathbf{g}_i^{\mathcal{K}^{1-a^*}_{\mathrm{Lft}(g^*)}}, \mathbf{h}_i^{\mathcal{K}^{1-a^*}_{\mathrm{Lft}(g^*)}}, \mathbf{g}_i^{\mathcal{K}^{1-b^*}_{\mathrm{Rgt}(g^*)}}, \mathbf{h}_i^{\mathcal{K}^{1-b^*}_{\mathrm{Rgt}(g^*)}}$.
- In the other case from the group elements $(\mathbf{g}', \mathbf{h}', \mathbf{g}'', \mathbf{h}'')$.

$\mathcal{A}$ will be given one of these two tuples, and it will use $\mathcal{A}^*$ to try and tell them apart. To provide a valid input for $\mathcal{A}^*$, $\mathcal{A}$ should also be given elements of the form $\mathbf{g}_j^{\mathcal{K}^{1-a^*}_{\mathrm{Lft}(g^*)}}, \mathbf{h}_j^{\mathcal{K}^{1-a^*}_{\mathrm{Lft}(g^*)}}, \mathbf{g}_j^{\mathcal{K}^{1-b^*}_{\mathrm{Rgt}(g^*)}}, \mathbf{h}_j^{\mathcal{K}^{1-b^*}_{\mathrm{Rgt}(g^*)}}$, for all j which are used by EC-gateGb in gates sharing an input with g^*. (By design, these values are all distinct from $\mathbf{g}_i, \mathbf{h}_i$.) It can be verified that these inputs allow $\mathcal{A}$ to faithfully sample the inputs for $\mathcal{A}_3^*$ in either of the two experiments $\mathrm{Exp}^{*\mathrm{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^{**}}$ and $\mathrm{Exp}^{\mathrm{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^*}(0)$, depending on which of the above mentioned 4-tuples that $\mathcal{A}$ gets.

Hence, by Lemma 6.5

$$\begin{aligned} & \left| \Pr[\mathrm{Exp}^{*\mathrm{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^{**}} = 1] - \Pr[\mathrm{Exp}^{\mathrm{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^*}(1) = 1] \right| \\ \leq & 4\mathrm{Adv}^{\mathrm{altDDH}^*}_{\mathbb{G},\mathcal{A}} + 64/\sqrt{q} \end{aligned}$$

Hence, the total advantage

$$\mathrm{Adv}^{\mathrm{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^*} \leq \frac{64 + 9/\beta^2}{\sqrt{q}} + 4\mathrm{Adv}^{DDH}_{\mathbb{G},\mathcal{A}}$$

which is negligible under the DDH assumption. □

7 Zero-knowledge Proof of Correct Garbling

We use a ZK-SNARK system that supports arithmetic circuits with lookup gates [12, 13]. Below we explain the low-level details of this implementation, and the costs we incur (summarized in Table 1).

Exponentiation using lookups. We require 8 elliptic curve exponentiations per gate (two group generators each exponentiated to 4 labels). Elliptic curve exponentiation is very expensive in terms of number of arithmetic gates required. But we can use lookup gates to reduce the number of gates required substantially. Consider computing $\mathbf{g}^x$ where $\mathbf{g} \in \mathbb{G}$ and $x \in \mathbb{F}_q$ where q. fits into 256 bits. We write $x = \sum_{k=0}^{15} x_k$ where each x_k has all but one of its 16 blocks filled with zeros. Since the value of $\mathbf{g}$ is pre-determined, we can have the exponents of the form $\mathbf{g}^{x_k}$ pre-computed and stored in a table, so that $\mathbf{g}^x = \prod_{k=0}^{15} \mathbf{g}^{x_k}$ where we lookup the value of $\mathbf{g}^{x_k}$ from the lookup table. (We do not need to check that the split is indeed valid, since the fact that it can be looked up in the table is good enough.) The number of entries in this table, i.e., the total number of possible x_k values, is $16 \times 2^{16} = 2^{20}$.

Now, expressing $x = \sum_{k=0}^{15} x_k$ requires 15 addition gates. Since we need to split 4 labels per gate, the total number of addition gates for the splits is $15 \times 4 = 60$. Also, the total number of lookups per gate, for the 8 exponentiations is $8 \times 16 = 128$. The remaining cost for verifying $\mathbf{g}^x$ is of 15 group operations in $\prod_{k=0}^{15} \mathbf{g}^{x_k}$, which we consider next.

Group Operations: Plonk [14] is a natural choice for a ZK-SNARK to be used with a lookup table. We instantiate it using the pairing friendly curve BN128 and Baby Jubjub Edward curve [30] defined over the scalar field of BN128. The following is the formula for the group operation in the curve:

$$(x_3, y_3) = (x_1, y_1) + (x_2, y_2) = \left(\frac{x_1 y_2 + x_2 y_1}{1 + d x_1 x_2 y_1 y_2}, \frac{y_1 y_2 - x_1 x_2}{1 - d x_1 x_2 y_1 y_2} \right)$$

where (x_1, y_1) and (x_2, y_2) are the coordinates of the two group points and d is the "twist." The following gates in the Plonk system can be used for verifying the group operation formula:

$$\begin{aligned} x' &= x_1 x_2 & y' &= y_1 y_2 & z_1 &= x_1 y_2 \\ z_2 &= x_2 y_1 & z_3 &= 1 + d x' y' & z_4 &= 2y_3 - y_3 z_3 \\ z_4 &= y' - x' & z_5 &= x_3 z_3 & z_5 &= z_1 + z_2 \end{aligned}$$

Each group operation computed as above uses 9 binary gates – 7 multiplication gates (which are allowed to include addition too, as in Plonk) and 2 addition gates. So, the total number of addition and multiplication gates for the 8 exponentiations in each gate is $8 \times 15 \times 2 = 240$ and $8 \times 15 \times 7 = 840$, respectively.

Remaining costs: After computing the exponentiations, we must verify that the product of the matrix $\mathbf{M}$ with the vector $[\mathbf{a}\ \mathbf{b}\ \mathbf{c}]$ satisfies the condition required in EC-gateGb. Computing $\mathbf{M} = [A\ B\ C]$ involves 8 squarings (of r_b, s_b, t_b, u_b for $b \in \{0, 1\}$) for computing A, B, and further 16 multiplications for computing $C = A \odot B$. Computing $A\mathbf{a}$ and $B\mathbf{b}$ each involves a 8 multiplications and 6 additions (since rows repeat) while $C\mathbf{c}$ involves 16 multiplications and 12 additions; the overall matrix-vector multiplication $A\mathbf{a} + B\mathbf{b} + C\mathbf{c}$ uses 8 more additions. Overall, these costs come to $8 + 16 + 16 + 16 = 56$ multiplications and $12 + 12 + 8 = 32$ additions.

Hence, the total number of addition gates required per garbled gate is 60 + 240 + 32 = 332 and the total number of multiplication gates per garbled gate is 840 + 56 = 896.

8 Zero-Knowledge Proof for Oblivious Transfer

Recall that in the mkSol algorithm in Section 4, the garbler uses an OT protocol to transfer labels for each input wire of the circuit. The protocol is not secure unless the garbler proves the correctness of this step. The OT involves the garbler encrypting both the values, but the public keys are generated in a way that the evaluator can only decrypt one of them. Hence, the garbler simply has to prove that it correctly encrypted both the labels for each wire. One approach is to use an arbitrary OT protocol and generically prove that the encryptions are correct as well.

However, this approach requires that the garbler creates arithmetic circuit for the exponentiation of the public keys. This would require a huge number of gates and increase the prover time. Instead, we take the following approach:

- The garbler creates commitments to each label that can be opened by exponentiating fixed curve points. The circuit for this can use lookup tables for optimization.
- These commitments are shared to the verifier along with the OT messages.
- We create a separate proof using Bulletproof [9] to show that the OT correctly encrypts the committed values.

For OT, we can use the protocol of Peikert et. al. [27]. This protocol allows an OT of either of two points on an elliptic curve. Any input label must be a random label from $\mathbb{F}_q$. Given a random $\mathbf{u}, \mathbf{v} \leftarrow \mathbb{G}$, the receiver can obtain a label $\alpha \cdot \mathrm{x}(\mathbf{u}) + \beta \cdot \mathrm{x}(\mathbf{v})$ for uniformly chosen, but public $\alpha, \beta \leftarrow \mathbb{F}_q$. The method of OT is as described below -

(1) During initial setup, a CRS $(\mathbf{g}_0, \mathbf{h}_0, \mathbf{g}_1, \mathbf{h}_1) \in \mathbb{G}^4$ is decided.
(2) For the input b of the receiver, the receiver chooses $y \leftarrow \mathbb{F}_q$ and sends $(\mathbf{r}, \mathbf{s}) = (\mathbf{g}_b^y, \mathbf{h}_b^y)$ to the sender.
(3) Let the senders inputs be $\mathbf{v}_0, \mathbf{v}_1 \in \mathbb{G}$. The sender chooses $s, t \leftarrow \mathbb{F}_q$ and computes $(\mathbf{e}_{b'}, \mathbf{f}_{b'}) = (\mathbf{g}_{b'}^s \mathbf{h}_{b'}^t, \mathbf{r}^s \mathbf{s}^t \mathbf{v}_{b'})$ for each $b' \in \{0, 1\}$.
(4) Receiver decrypts $\mathbf{v}_b = \mathbf{f}_b \mathbf{e}_b^{y^{-1}}$.

Using two such transfers, a random label of an input can be sent to the receiver. To commit the labels, we commit the curve points that construct the labels. For every label, two random coefficients α, β and generated and two random points $\mathbf{u}, \mathbf{v}$ are chosen. The label is then $\alpha \cdot \mathrm{x}(u) + \beta \cdot \mathrm{x}(\mathbf{v})$. To commit a curve point $\mathbf{w}$, we use ElGamal encryption. The garbler chooses a global private key x and a corresponding public key $(\mathbf{g}, \mathbf{x})$. The commitment of the curve point is then $(\mathbf{c}, \mathbf{d}) = (\mathbf{g}^y, \mathbf{x}^y \mathbf{w})$.

- **CRS:** Since we want to use bulletproof and we need public verifiability, we need a different CRS for each copy of the OT. For each input $i \in \{1..n\}$, two curve points $\mathbf{u}_i^b, \mathbf{v}_i^b$ need to be sent through OT for each input value $b \in 0, 1$. The CRS for the ith input is $(\mathbf{g}_{i0}, \mathbf{h}_{i0}, \mathbf{g}_{i1}, \mathbf{h}_{i1}, \mathbf{g}'_{i0}, \mathbf{g}'_{i1}, \mathbf{h}'_{i0}, \mathbf{h}'_{i1}) \leftarrow \mathbb{G}^8$. We also have four randomly generated ElGamal public key pairs $(\mathbf{l}_{i0}, \mathbf{p}_{i0})$, $(\mathbf{l}_{i1}, \mathbf{p}_{i1})$, $(\mathbf{l}'_{i0}, \mathbf{p}'_{i0})$, $(\mathbf{l}'_{i1}, \mathbf{p}'_{i1}))$ for each input i. Note that the private key for the ElGamal encryption is never known to anyone.

  It is possible to use the same set of points for the public keys that we use for constructing gates. That way we can share the same lookup tables for exponentiation.
- **Oblivious Transfer:** The oblivious transfer of each curve point is done using the PVW protocol.
- **Commitment to the labels:** The commitments to the label is is achieved by ElGamal encrypting each of the points separately.
  (1) Each label is generated in the following manner - we have $(\alpha_{i0}, \beta_{i0}, \alpha_{i1}, \beta_{i1}) \leftarrow \mathbb{F}_q$. The label $\mathcal{K}_i^b = \alpha_{ib} \cdot \mathrm{x}(\mathbf{u}_{ib}) + \beta_{ib} \cdot \mathrm{x}(\mathbf{v}_{ib})$ is generated for each input i and $b \in \{0, 1\}$.
  (2) The points $\mathbf{u}_{i0}, \mathbf{u}_{i1}, \mathbf{v}_{i0}, \mathbf{v}_{i1}$ are encrypted using ElGamal. The garbler chooses $r_{i0}, r_{i1}, r'_{i0}, r'_{i1}$ from $\mathbb{F}_q$. Then computes $(\mathbf{c}_{ib}, \mathbf{d}_{ib}) = (\mathbf{l}_{ib}^{r_{ib}}, \mathbf{p}_{ib}^{r_{ib}} \mathbf{u}_{ib})$ and $(\mathbf{c}'_{ib}, \mathbf{d}'_{ib}) = (\mathbf{l}'^{\,r_{ib}}_{ib}, \mathbf{p}'^{\,r_{ib}}_{ib} \mathbf{v}_{ib})$. The ElGamal encryptions are shared with the receiver and are used in the zero-knowledge proof for correctness (r_{ib}, r'_{ib} are used as witness).
- **Oblivious Transfer:** We use the PVW protocol mentioned above for OT. We can denote the randomness used as s_{ib}, t_{ib} for transferring $\mathbf{u}_{ib}$ and s'_{ib}, t'_{ib} for transferring $\mathbf{v}_{ib}$.
- **Proof of correctness of the OT:** We use the CRS double as the CRS for Bulletproof. For each input i, the garbler needs to prove that there exists r_{ib}, s_{ib}, t_{ib} such that $\mathbf{c}_{ib} = \mathbf{l}_{ib}^{r_{ib}}$, $\mathbf{f}_{ib} \mathbf{d}_{ib}^{-1} = \mathbf{r}_i^{s_i} s_i^{t_i} \mathbf{p}_{ib}^{-r_{ib}}$, $\mathbf{e}_{ib} = \mathbf{g}_{ib}^{s_{ib}} \mathbf{h}_{ib}^{t_{ib}}$ and also the same relationships for $\mathbf{v}_{ib}$.

9 Conclusion

The main *conceptual contribution* of this work is to introduce Secret-Carrying Puzzles (OxSP) as a versatile tool that brings the power of secure 2-party computation to a blockchain setting. We showed that OxSP can be instantiated using Garbled Circuits and ZK proofs. This construction is fairly efficient, as long as the ZK proofs can be generated efficiently.

To that end, we present a novel garbling scheme optimized to minimize the number of arithmetic gates required to compute each gate. The main *technical contribution* of this work is to design this garbling scheme and to show how to generate ZK proofs for it efficiently.

We leave it for future work to support garbled circuit optimization techniques (see, e.g., [19]) that have been developed without support for proofs.

References

[1] Jackson Abascal, Mohammad Hossein Faghihi Sereshgi, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. 2020. Is the Classical GMW Paradigm Practical? The Case of Non-Interactive Actively Secure 2PC. In *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020,* Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna (Eds.). ACM, 1591–1605. https://doi.org/10.1145/3372297.3423366

[2] Aysajan Abidin, Erik Pohle, and Bart Preneel. 2023. Arithmetic Circuit Implementations of S-boxes for SKINNY and PHOTON in MPC. Cryptology ePrint Archive, Paper 2023/1426. https://eprint.iacr.org/2023/1426 https://eprint.iacr.org/2023/1426.

[3] Anasuya Acharya, Carmit Hazay, Vladimir Kolesnikov, and Manoj Prabhakaran. 2024. Malicious Security for SCALES - Outsourced Computation with Ephemeral Servers. In *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part IX (Lecture Notes in Computer Science, Vol. 14928),* Leonid Reyzin and Douglas Stebila (Eds.). Springer, 3–38. https://doi.org/10.1007/978-3-031-68400-5_1

[4] Arash Afshar, Payman Mohassel, Benny Pinkas, and Ben Riva. 2014. Non-Interactive Secure Computation Based on Cut-and-Choose. In *Advances in Cryptology – EUROCRYPT 2014,* Phong Q. Nguyen and Elisabeth Oswald (Eds.).

Springer, Berlin, Heidelberg, 387–404. https://doi.org/10.1007/978-3-642-55220-5_22

[5] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. 2023. Ligero: lightweight sublinear arguments without a trusted setup. *Des. Codes Cryptography* 91, 11 (July 2023), 3379–3424. https://doi.org/10.1007/s10623-023-01222-8

[6] Marcin Andrychowicz, Stefan Dziembowski, Daniel Malinowski, and Lukasz Mazurek. 2014. Secure Multiparty Computations on Bitcoin. In *2014 IEEE Symposium on Security and Privacy, SP 2014, Berkeley, CA, USA, May 18-21, 2014*. IEEE Computer Society, 443–458. https://doi.org/10.1109/SP.2014.35

[7] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. 2012. Foundations of garbled circuits. In *the ACM Conference on Computer and Communications Security, CCS'12, Raleigh, NC, USA, October 16-18, 2012*, Ting Yu, George Danezis, and Virgil D. Gligor (Eds.). ACM, 784–796. https://doi.org/10.1145/2382196.2382279

[8] Chris Brzuska and Sabine Oechsner. 2023. A State-Separating Proof for Yao's Garbling Scheme. In *2023 IEEE 36th Computer Security Foundations Symposium (CSF)*. 137–152. https://doi.org/10.1109/CSF57540.2023.00009

[9] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. 2018. Bulletproofs: Short Proofs for Confidential Transactions and More. In *39th IEEE Symposium on Security and Privacy 2018*.

[10] Christian Cachin, Jan Camenisch, Joe Kilian, and Joy Müller. 2000. One-Round Secure Computation and Secure Autonomous Mobile Agents. In *Automata, Languages and Programming*, Ugo Montanari, José D. P. Rolim, and Emo Welzl (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 512–523.

[11] Christopher Cordi, Michael P. Frank, Kasimir Gabert, Carollan Helinski, Ryan C. Kao, Vladimir Kolesnikov, Abrahim Ladha, and Nicholas Pattengale. 2022. Auditable, Available and Resilient Private Computation on the Blockchain via MPC. In *Cyber Security, Cryptology, and Machine Learning*, Shlomi Dolev, Jonathan Katz, and Amnon Meisels (Eds.). Springer International Publishing, Cham, 281–299.

[12] Liam Eagen, Dario Fiore, and Ariel Gabizon. 2022. Cq: Cached Quotients for Fast Lookups. *IACR Cryptol. ePrint Arch.* 2022/1763 (2022).

[13] Ariel Gabizon and Zachary J. Williamson. 2020. Plookup: A Simplified Polynomial Protocol for Lookup Tables. *IACR Cryptol. ePrint Arch.* 2020/315 (2020).

[14] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. 2019. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive Arguments of Knowledge. *IACR Cryptol. ePrint Arch.* 2019/953 (2019).

[15] Hongmin Gao, Zhaofeng Ma, Shoushan Luo, and Zhen Wang. 2019. BFR-MPC: A Blockchain-Based Fair and Robust Multi-Party Computation Scheme. *IEEE Access* 7 (2019), 110439–110450. https://doi.org/10.1109/ACCESS.2019.2934147

[16] Steven Goldfeder. 2017. A Boolean Circuit for SHA-256. https://stevengoldfeder.com/projects/circuits/sha2circuit.html.

[17] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. 2023. Poseidon2: A Faster Version of the Poseidon Hash Function. Cryptology ePrint Archive, Paper 2023/323. https://eprint.iacr.org/2023/323

[18] Shay Gueron, Yehuda Lindell, Ariel Nof, and Benny Pinkas. 2018. Fast Garbling of Circuits Under Standard Assumptions. *Journal of Cryptology* 31, 3 (July 2018), 798–844. https://doi.org/10.1007/s00145-017-9271-y

[19] David Heath. 2024. Efficient Arithmetic in Garbled Circuits. In *Advances in Cryptology – EUROCRYPT 2024*. Springer, Cham, 3–31. https://doi.org/10.1007/978-3-031-58740-5_1

[20] Omer Horvitz and Jonathan Katz. 2007. Universally-Composable Two-Party Computation in Two Rounds. In *Advances in Cryptology - CRYPTO 2007*, Alfred Menezes (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 111–129.

[21] Yan Huang, Jonathan Katz, Vladimir Kolesnikov, Ranjit Kumaresan, and Alex J. Malozemoff. 2014. Amortizing Garbled Circuits. In *Advances in Cryptology – CRYPTO 2014*, Juan A. Garay and Rosario Gennaro (Eds.). Springer, Berlin, Heidelberg, 458–475. https://doi.org/10.1007/978-3-662-44381-1_26

[22] Yuval Ishai, Dakshita Khurana, Amit Sahai, and Akshayaram Srinivasan. 2023. Black-Box Reusable NISC with Random Oracles. In *Advances in Cryptology – EUROCRYPT 2023*, Carmit Hazay and Martijn Stam (Eds.). Springer Nature Switzerland, Cham, 68–97.

[23] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, Manoj Prabhakaran, and Amit Sahai. 2011. Efficient Non-interactive Secure Computation. In *Advances in Cryptology - EUROCRYPT 2011 - 30th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tallinn, Estonia, May 15-19, 2011. Proceedings (Lecture Notes in Computer Science, Vol. 6632)*, Kenneth G. Paterson (Ed.). Springer, 406–425. https://doi.org/10.1007/978-3-642-20465-4_23

[24] Shan Jin, Yong Li, Xi Chen, and Ruxian Li. 2023. Blockchain Based Publicly Auditable Multi-party Computation with Cheater Detection. In *Information and Communications Security*, Ding Wang, Moti Yung, Zheli Liu, and Xiaofeng Chen (Eds.). Springer Nature, Singapore, 608–626. https://doi.org/10.1007/978-981-99-7356-9_36

[25] Yehuda Lindell. 2016. Fast Cut-and-Choose-Based Protocols for Malicious and Covert Adversaries. *Journal of Cryptology* 29, 2 (April 2016), 456–490. https://doi.org/10.1007/s00145-015-9198-0

[26] Yehuda Lindell and Ben Riva. 2014. Cut-and-Choose Yao-Based Secure Computation in the Online/Offline and Batch Settings. In *Advances in Cryptology – CRYPTO 2014*, Juan A. Garay and Rosario Gennaro (Eds.). Springer, Berlin, Heidelberg, 476–494. https://doi.org/10.1007/978-3-662-44381-1_27

[27] Chris Peikert, Vinod Vaikuntanathan, and Brent Waters. 2008. A Framework for Efficient and Composable Oblivious Transfer. In *Proceedings of the 28th Annual Conference on Cryptology: Advances in Cryptology* (Santa Barbara, CA, USA) *(CRYPTO 2008)*. Springer-Verlag, Berlin, Heidelberg, 554–571. https://doi.org/10.1007/978-3-540-85174-5_31

[28] Minhye Seo. 2021. Fair and Secure Multi-Party Computation with Cheater Detection. *Cryptography* 5, 3 (Sept. 2021), 19. https://doi.org/10.3390/cryptography5030019

[29] Xiaoqiang Sun, F. Richard Yu, Peng Zhang, Zhiwei Sun, Weixin Xie, and Xiang Peng. 2021. A Survey on Zero-Knowledge Proof in Blockchain. *IEEE Network* 35, 4 (July 2021), 198–205. https://doi.org/10.1109/MNET.011.2000473

[30] Barry WhiteHat, Jordi Baylina, and Marta Belles. 2016. Baby Jubjub Elliptic Curve. https://docs.iden3.io/publications/pdfs/Baby-Jubjub.pdf.

[31] Yuhan Yang, Jing Wu, Chengnian Long, Wei Liang, and Yi-Bing Lin. 2022. Blockchain-Enabled Multiparty Computation for Privacy Preserving and Public Audit in Industrial IoT. *IEEE Transactions on Industrial Informatics* 18, 12 (2022), 9259–9267. https://doi.org/10.1109/TII.2022.3177630

[32] Ruiyu Zhu, Changchang Ding, and Yan Huang. 2019. Efficient Publicly Verifiable 2PC over a Blockchain with Applications to Financially-Secure Computations. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. ACM, London United Kingdom, 633–650. https://doi.org/10.1145/3319535.3363215

A Security of Gate-by-Gate Garbling

In this section we proof Theorem 5.2, which states that a gate-by-gate garbling scheme that uses a GateSim secure gate garbling is Sim secure overall.

Proof of Theorem 5.2. Let $\mathcal{S}^*$ denote the simulator guaranteed by Definition 5.1 for gateGb. We shall use it to define a pair of simulators $(\mathcal{S}_1, \mathcal{S}_2)$ as required in Definition 3.2.

$\mathcal{S}_1(1^k, \mathsf{Topo}(\mathsf{C}))$ proceeds as follows (recall that $\mathsf{Topo}(\mathsf{C})$ is the DAG defining the circuit C, and hence includes the information to specify the *sets* C, C_{in} and C_{out}, and the functions $\mathsf{Lft}(\cdot)$ and $\mathsf{Rgt}(\cdot)$, but not the type of the gates).

(1) $\mathsf{par} \leftarrow \mathsf{setup}(1^k, \mathsf{Topo}(\mathsf{C}))$, and for each $g \in \mathsf{C}$, $\widetilde{\mathcal{K}}_g \leftarrow \mathsf{sample}(\mathsf{par})$.
(2) Let $\mathbf{X} := \{\widetilde{\mathcal{K}}_g\}_{g \in \mathsf{C}_{\text{in}}}$
(3) Let $F := (\{\mathbf{\Psi}_g\}_{g \in \mathsf{C} \setminus \mathsf{C}_{\text{in}}}, \mathsf{par})$, where
$$\mathbf{\Psi}_g \leftarrow \mathcal{S}^*(\widetilde{\mathcal{K}}_{\mathsf{Lft}(g)}, \widetilde{\mathcal{K}}_{\mathsf{Rgt}(g)}, \widetilde{\mathcal{K}}_g, g, \mathsf{par}).$$
(4) Return $(F, \mathbf{X})$.

The simulator $\mathcal{S}_2(F, \mathbf{Y}, \mathbf{y})$ proceeds as follows:

(1) Parse $\mathbf{Y}$ as $\{\mathbf{Y}_g\}_{g \in T}$, and $\mathbf{y}$ as $\{\mathbf{y}_g\}_{g \in T}$, where each $\mathbf{y}_g \in \{0,1\}$ and T is a set of indices.
(2) For each $g \in T$, let
$$\mathcal{K}_g^{\mathbf{y}_g} := \mathbf{Y}_g \qquad \mathcal{K}_g^{1-\mathbf{y}_g} \leftarrow \mathsf{sample}(\mathsf{par})$$
where par is obtained from F.
(3) Define d such that for each $g \in T$ and $b \in \{0,1\}$, $d(g, \mathcal{K}_g^b) = b$. Return d.

To show that the simulation meets the requirements of Definition 3.2, we shall consider a sequence of hybrid experiments. Given a PPT adversary $\mathcal{A}$, let $N = N(k)$ be a polynomial upper bound on the number of non-input gates in the circuit that it outputs in the first stage. To analyze the advantage of this adversary against the above simulation, we shall define hybrid experiments using a hybrid garbling-encoding algorithm $\widetilde{\mathsf{Gb}}_\ell$ (for $\ell = 0$ to N) which carries out garbling and encoding as in the real scheme, but then replaces garbling of the first ℓ non-input gates (in the topologically sorted order) with gates simulated using $\mathcal{S}^*$ and only the "active" labels. Then we observe that the experiment using $\widetilde{\mathsf{Gb}}_N$ is identically distributed as the experiment using $(\mathcal{S}_1, \mathcal{S}_2)$ defined above (even though syntactically the garbling and encoding in the former uses the input and the gate types). To complete the proof, we shall argue that distinguishing between adjacent hybrids using $\widetilde{\mathsf{Gb}}_{\ell-1}$ and $\widetilde{\mathsf{Gb}}_\ell$ amounts to breaking the security guarantee of $\mathcal{S}^*$. Details follow.

Hybrid Garbling and Encoding Scheme $\widetilde{\mathsf{Gb}}_\ell(1^k, \mathsf{C}, \mathbf{x})$.
Let $\mathsf{C} \setminus \mathsf{C}_{\text{in}} = \{g_1, \ldots, g_N\}$ be ordered topologically (so that for all $i \in [N]$, $\mathsf{Lft}(g_i) = g_j$ implies $j < i$ and similarly for $\mathsf{Rgt}(g_i)$).
Also, for all $g \in \mathsf{C}$, let $w_g \in \{0,1\}$ denote the value of the gate (wire) g when $\mathsf{C}(\mathbf{x})$ is evaluated.

(1) Let $\mathsf{par} \leftarrow \mathsf{setup}(\mathsf{Topo}(\mathsf{C}))$ and $(F, e, d) \leftarrow \mathsf{Gb}(\mathsf{C}, \mathsf{par})$, where $F = (\{\mathbf{\Psi}_{g_i}\}_{i=1}^N, \mathsf{par})$.
(2) For $i \le \ell$, let
$$\mathbf{\Psi}'_{g_i} := \mathcal{S}^*(\mathcal{K}_{\mathsf{Lft}(g_i)}^{w_{\mathsf{Lft}(g_i)}}, \mathcal{K}_{\mathsf{Rgt}(g_i)}^{w_{\mathsf{Rgt}(g_i)}}, \mathcal{K}_{g_i}^{w_{g_i}}, g, \mathsf{par})$$
where the labels $\{\mathcal{K}_g^b\}_{g \in \mathsf{C}, b \in \{0,1\}}$ are those generated by Gb in the previous step.
(3) Output $(F', \mathbf{X}, d)$ where $F' = (\{\mathbf{\Psi}'_{g_i}\}_{i=1}^\ell, \{\mathbf{\Psi}_{g_i}\}_{i=\ell+1}^N, \mathsf{par})$, $\mathbf{X} = \mathsf{En}(e, \mathbf{x})$.

Then we claim the following:

Claim 1. *For any circuit* C *with* N *non-input gates, and any valid input* $\mathbf{x}$ *for* C*, the following equalities of distributions hold:*

$$\{(F, \mathbf{X}, d)\}_{(F,\mathbf{X},d) \leftarrow \widetilde{\mathsf{Gb}}_0(1^k, \mathsf{C}, \mathbf{x})} \equiv \{(F, \mathbf{X}, d)\}_{\substack{(F,e,d) \leftarrow \mathsf{Gb}(1^k, \mathsf{C}) \\ \mathbf{X} \leftarrow \mathsf{En}(e, \mathbf{x})}} \tag{3}$$

$$\{(F, \mathbf{X}, d)\}_{(F,\mathbf{X},d) \leftarrow \widetilde{\mathsf{Gb}}_N(1^k, \mathsf{C}, \mathbf{x})} \equiv \{(F, \mathbf{X}, d)\}_{\substack{(F,\mathbf{X}) \leftarrow \mathcal{S}_1(1^k, \mathsf{Topo}(\mathsf{C})) \\ d \leftarrow \mathcal{S}_2(F, \mathsf{Ev}(F, \mathbf{X}), \mathsf{C}(\mathbf{x}))}} \tag{4}$$

Proof. The first equality directly follows from the definition of $\widetilde{\mathsf{Gb}}_0$ (step 2 in $\widetilde{\mathsf{Gb}}_\ell$ has no effect and $F' = F$, when $\ell = 0$). To see the second equality, note that in $\widetilde{\mathsf{Gb}}_N$ none of the labels $\mathcal{K}_g^{1-w_g}$ are used, except for the output gates g, where they appear in the decoding information d; as such, for each non-output gate only one label $\widetilde{\mathcal{K}}_g$ is used (for input gates, this label appears directly in $\mathbf{X}$); for each output gate g, two labels $\mathcal{K}_g^0$ and $\mathcal{K}_g^1$ are recorded in d, and $\widetilde{\mathcal{K}}_g$ is set to $\mathcal{K}_g^{w_g}$, where w_g is the bit of $\mathsf{C}(\mathbf{x})$ corresponding to the output gate g. It can then be seen that the execution of $(\mathcal{S}_1, \mathcal{S}_2)$ results in exactly the same distribution for $(F, \mathbf{X}, d)$. □ □

We also use the following claim.

Claim 2. *For any two-stage PPT adversary* $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ *as in Definition 3.2, which outputs circuits of size at most* N*, and for each* $\ell \in [N]$*, there exists a three-stage PPT adversary* $\mathcal{A}^*$ *as in Definition 5.1 such that*

$$\left| \Pr_{\substack{(\sigma, \mathsf{C}, \mathbf{x}) \leftarrow \mathcal{A}_1 \\ (F, \mathbf{X}, d) \leftarrow \widetilde{\mathsf{Gb}}_{\ell-1}(1^k, \mathsf{C}, \mathbf{x})}} [\mathcal{A}_2(\sigma, F, \mathbf{X}, d) = 1] - \Pr_{\substack{(\sigma, \mathsf{C}, \mathbf{x}) \leftarrow \mathcal{A}_1 \\ (F, \mathbf{X}, d) \leftarrow \widetilde{\mathsf{Gb}}_\ell(1^k, \mathsf{C}, \mathbf{x})}} [\mathcal{A}_2(\sigma, F, \mathbf{X}, d) = 1] \right| = \mathsf{Adv}^{\mathsf{gateGb}}_{\mathsf{C}, \mathcal{A}^*, \mathcal{S}^*} \tag{5}$$

Proof. Fix a two-stage adversary $\mathcal{A}$ that outputs circuits with at most N (non-input) gates, and $\ell \in [N]$. We consider a subcircuit C'_ℓ of C that contains only the gates that are not simulated by $\widetilde{\mathsf{Gb}}_\ell$ combined with $\mathsf{C}_{in,\ell}$ that is the set of open input gates of the C'_ℓ that are not output of any gate in C'_ℓ. We define C_ℓ to be $\mathsf{C}'_\ell \cup \mathsf{C}_{\ell,\ell}$. Then we sketch the description of $\mathcal{A}^* = (\mathcal{A}_1^*, \mathcal{A}_2^*, \mathcal{A}_3^*)$ as stated in the claim.

- $\mathcal{A}_1^*$ runs $\mathcal{A}_1$ to obtain $\sigma_1^* := (\sigma, \mathsf{C}, \mathbf{x})$ and outputs $(\sigma_1^*, \mathsf{C}_\ell)$, where C_ℓ is obtained from C as defined above.
- $\mathcal{A}_2^*$, on input $(\sigma_1^*, \mathsf{par})$, outputs (σ_2^*, g^*, u, v) defined as follows: $g^* := g_\ell$ according to the topological sorting of $\mathsf{C} \setminus \mathsf{C}_{\mathsf{in}}$ used in defining $\widetilde{\mathsf{Gb}}_\ell$; $u = w_{\mathsf{Lft}(g^*)}$ and $v = w_{\mathsf{Rgt}(g^*)}$, where w_g denotes the value on the wire g when C is evaluated on input $\mathbf{x}$.
- $\mathcal{A}_3^*$, on input $\big(\sigma_2^*, \{\mathbf{\Psi}_g\}_{g \in \mathsf{C}_\ell \setminus \mathsf{C}_{in,\ell}}, \{\mathcal{K}_g^x\}_{(g,x) \notin \{(\mathsf{Lft}(g^*),1-u),(\mathsf{Rgt}(g^*),1-v)\}}\big)$, produces $(F, \mathbf{X}, d)$ as described below, and outputs $\mathcal{A}_2(\sigma, F, \mathbf{X}, d)$.
  - For every gate in $\mathsf{C} \setminus \mathsf{C}_\ell$, sample the active label.
  - $F = (\{\mathbf{\Psi}'_{g_i}\}_{i=1}^{\ell-1}, \{\mathbf{\Psi}_{g_i}\}_{i=\ell}^{N}, \mathsf{par})$, where $\mathbf{\Psi}'_g = \mathcal{S}^*(\mathcal{K}_{\mathsf{Lft}(g)}^{w_{\mathsf{Lft}(g)}}, \mathcal{K}_{\mathsf{Rgt}(g)}^{w_{\mathsf{Rgt}(g)}}, \mathcal{K}_g^{w_g} g, \mathsf{par})$. The set of simulated garbled gates $\mathbf{\Psi}'_g$ can be computed using the available labels, since the two missing labels are of the form $\mathcal{K}_g^{1-w_g}$ and such labels are not needed for the simulation.
  - $\mathbf{X} = \{\mathcal{K}_g^{w_g}\}_{g \in \mathsf{C}_{\mathsf{in}}}$ can also be computed from the given labels and the labels just freshly sampled.
  - Finally, d includes $\{\mathcal{K}_g^0, \mathcal{K}_g^1\}_{g \in \mathsf{C}_{\mathsf{out}}}$. These labels are all available, since $\mathsf{Lft}(g^*), \mathsf{Rgt}(g^*) \notin \mathsf{C}_{\mathsf{out}}$ (since the output gates are sinks, and do not appear as inputs to any other gates).

The above construction is such that the output of $\mathsf{Exp}^{\mathsf{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^*}(0)$ (wherein the garbled circuit received by $\mathcal{A}^*$ does not include any simulated gate) is identical to the output of the experiment in which $\mathcal{A}$ interacts with $\widetilde{\mathsf{Gb}}_{\ell-1}$. On the other hand, in $\mathsf{Exp}^{\mathsf{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^*}(1)$ (wherein g_ℓ is garbled using the simulator $\mathcal{S}^*$), the output is identical to when $\mathcal{A}$ interacts with $\widetilde{\mathsf{Gb}}_\ell$. The claim follows. □

Given the above two claims, the advantage of any two-stage adversary $\mathcal{A}$ in Definition 3.2 is equal to the following:

$$\left| \Pr_{\substack{(\sigma,\mathsf{C},\mathbf{x}) \leftarrow \mathcal{A}_1 \\ (F,e,d) \leftarrow \mathsf{Gb}(1^k,\mathsf{C}) \\ \mathsf{X} \leftarrow \mathsf{En}(e,\mathbf{x})}} [\mathcal{A}_2(\sigma, F, \mathbf{X}, d) = 1] - \Pr_{\substack{(\sigma,\mathsf{C},\mathbf{x}) \leftarrow \mathcal{A}_1 \\ (F,\mathsf{X}) \leftarrow \mathcal{S}_1(1^k,\phi(\mathsf{C})) \\ d \leftarrow \mathcal{S}_2(F,\mathsf{Ev}(F,\mathsf{X}),\mathsf{C}(\mathbf{x}))}} [\mathcal{A}_2(\sigma, F, \mathbf{X}, d) = 1] \right|$$

By (3) and (4), this equals:

$$\left| \Pr_{\substack{(\sigma,\mathsf{C},\mathbf{x}) \leftarrow \mathcal{A}_1 \\ (F,\mathsf{X},d) \leftarrow \widetilde{\mathsf{Gb}}_0(1^k,\mathsf{C},\mathbf{x})}} [\mathcal{A}_2(\sigma, F, \mathbf{X}, d) = 1] - \Pr_{\substack{(\sigma,\mathsf{C},\mathbf{x}) \leftarrow \mathcal{A}_1 \\ (F,\mathsf{X},d) \leftarrow \widetilde{\mathsf{Gb}}_N(1^k,\mathsf{C},\mathbf{x})}} [\mathcal{A}_2(\sigma, F, \mathbf{X}, d) = 1] \right|$$

This can be bounded by writing as a telescoping sum:

$$\begin{aligned} &\left| \sum_{\ell=1}^{N} \left(\Pr_{\substack{(\sigma,\mathsf{C},\mathbf{x}) \leftarrow \mathcal{A}_1 \\ (F,\mathsf{X},d) \leftarrow \widetilde{\mathsf{Gb}}_{\ell-1}(1^k,\mathsf{C},\mathbf{x})}} [\mathcal{A}_2(\sigma, F, \mathbf{X}, d) = 1] - \Pr_{\substack{(\sigma,\mathsf{C},\mathbf{x}) \leftarrow \mathcal{A}_1 \\ (F,\mathsf{X},d) \leftarrow \widetilde{\mathsf{Gb}}_{\ell}(1^k,\mathsf{C},\mathbf{x})}} [\mathcal{A}_2(\sigma, F, \mathbf{X}, d) = 1] \right) \right| \\ &\leq N \mathsf{Adv}^{\mathsf{gateGb}}_{C,\mathcal{A}^*,\mathcal{S}^*} \qquad \text{by (5)} \end{aligned}$$

where $\mathcal{A}^*$ is as guaranteed in the claim above. To conclude the proof, note that if $\mathcal{A}$ is PPT, so is $\mathcal{A}^*$, and hence the RHS is negligible, by GateSim security. As this holds for all PPT $\mathcal{A}$, Sim security holds. □

B ZK-friendly Garbling using a Random Oracle

In this section, we present an alternative GC construction that relies on a proof-friendly instantiation of a random oracle. Although our construction exploits Poseidon 2, compared to simply plugging in Poseidon 2 as a random oracle in a conventional GC construction, it achieves smaller garbled gates, without compromising the efficiency of the prover.

The input values are labeled with $\mathcal{K}^0_{W_R}$ and $\mathcal{K}^1_{W_R}$ for the right input and $\mathcal{K}^0_{W_L}$ and $\mathcal{K}^1_{W_L}$ for the left input wire. We use a hash function H modeled as a random oracle that outputs two field elements. The basic idea is to map the output of the random oracle to the output labels in a way so that the gate logic is followed. This mapping is done using a polynomial of three variables X, Y, Z of the form $G(X, Y, Z) = AZ + BX + CY + DZ^2$. Given the input values X, Y, we can compute $Z = H(X, Y)$ and then $O = G(X, Y, Z)$. To follow the gate logic, the following equation must hold -

$$\begin{bmatrix} z_{0,0} & \mathcal{K}^0_{\mathsf{Lft}(g)} & \mathcal{K}^0_{\mathsf{Rgt}(g)} & z^2_{0,0} \\ z_{0,1} & \mathcal{K}^0_{\mathsf{Lft}(g)} & \mathcal{K}^1_{\mathsf{Rgt}(g)} & z^2_{0,1} \\ z_{1,0} & \mathcal{K}^1_{\mathsf{Lft}(g)} & \mathcal{K}^0_{\mathsf{Rgt}(g)} & z^2_{1,0} \\ z_{1,1} & \mathcal{K}^1_{\mathsf{Lft}(g)} & \mathcal{K}^1_{\mathsf{Rgt}(g)} & z^2_{1,1} \end{bmatrix} \begin{bmatrix} A \\ B \\ C \\ D \end{bmatrix} = \begin{bmatrix} \mathcal{K}_g^{\mathsf{op}_g(0,0)} \\ \mathcal{K}_g^{\mathsf{op}_g(0,1)} \\ \mathcal{K}_g^{\mathsf{op}_g(1,0)} \\ \mathcal{K}_g^{\mathsf{op}_g(1,1)} \end{bmatrix}$$

where $z_{i,j} = H(\mathcal{K}^i_{\mathsf{Lft}(g)}, \mathcal{K}^j_{\mathsf{Rgt}(g)})$ for $i, j \in \{0, 1\}$

Hence, we define the function gateGb as follows -

$\mathsf{gateGb}(\mathsf{op}_g, \mathcal{K}^0_{\mathsf{Lft}(g)}, \mathcal{K}^1_{\mathsf{Lft}(g)}, \mathcal{K}^0_{\mathsf{Rgt}(g)}, \mathcal{K}^1_{\mathsf{Rgt}(g)}, \mathcal{K}^0_g, \mathcal{K}^1_g, g, \mathsf{par})$

(1) If the matrix

$$\mathcal{M} = \begin{bmatrix} z_{0,0} & \mathcal{K}^0_{\mathsf{Lft}(g)} & \mathcal{K}^0_{\mathsf{Rgt}(g)} & z^2_{0,0} \\ z_{0,1} & \mathcal{K}^0_{\mathsf{Lft}(g)} & \mathcal{K}^1_{\mathsf{Rgt}(g)} & z^2_{0,1} \\ z_{1,0} & \mathcal{K}^1_{\mathsf{Lft}(g)} & \mathcal{K}^0_{\mathsf{Rgt}(g)} & z^2_{1,0} \\ z_{1,1} & \mathcal{K}^1_{\mathsf{Lft}(g)} & \mathcal{K}^1_{\mathsf{Rgt}(g)} & z^2_{1,1} \end{bmatrix}$$

where $z_{i,j} = H(\mathcal{K}^i_{\mathsf{Lft}(g)}, \mathcal{K}^j_{\mathsf{Rgt}(g)})$ for $i, j \in \{0, 1\}$ is not invertible, return $\perp$

(2) Otherwise compute -

$$\begin{bmatrix} A_g \\ B_g \\ C_g \\ D_g \end{bmatrix} = \mathcal{M}^{-1} \begin{bmatrix} \mathcal{K}_g^{\mathrm{op}_g(0,0)} \\ \mathcal{K}_g^{\mathrm{op}_g(0,1)} \\ \mathcal{K}_g^{\mathrm{op}_g(1,0)} \\ \mathcal{K}_g^{\mathrm{op}_g(1,1)} \end{bmatrix}$$

(3) $A_g = 0$, return $\perp$.
(4) return (A_g, B_g, C_g, D_g)

B.0.1 Security.

Theorem B.1. *The garbling algorithm from random oracle is correct.*

Proof. The algorithm fails for every gate if either $\mathcal{M}$ is not invertible or $A_g = 0$. The determinant of $\mathcal{M}$ is a polynomial in 8 variables of maximum degree 3. It is not the zero polynomial since the variable $H(\mathcal{K}^0_{\mathrm{Lft}(g)}, \mathcal{K}^0_{\mathrm{Rgt}(g)})$ appears only once. Hence, due to the Schwartz–Zippel lemma, the maximum probability of the determinant being zero is $3/q$.

Now, we have

$$\begin{bmatrix} \mathcal{K}_g^{\mathrm{op}_g(0,0)} \\ \mathcal{K}_g^{\mathrm{op}_g(0,1)} \\ \mathcal{K}_g^{\mathrm{op}_g(1,0)} \\ \mathcal{K}_g^{\mathrm{op}_g(1,1)} \end{bmatrix} = \mathcal{M} \begin{bmatrix} A_g \\ B_g \\ C_g \\ D_g \end{bmatrix}$$

The probability of $\mathcal{K}_g^{\mathrm{op}_g(0,0)}$ being zero is $1/q$. Otherise, since, $\mathcal{K}_g^{\mathrm{op}_g(0,0)} \neq 0$, we can eliminate the bottom three rows of the left side by row operation. If then $A_g = 0$, we must have the following -

$$\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \mathcal{M}' \begin{bmatrix} B_g \\ C_g \\ D_g \end{bmatrix}$$

where,

$$\mathcal{M}' = \begin{bmatrix} \mathcal{K}^0_{\mathrm{Lft}(g)} - a \cdot \mathcal{K}^0_{\mathrm{Lft}(g)} & \mathcal{K}^1_{\mathrm{Rgt}(g)} - a \cdot \mathcal{K}^0_{\mathrm{Rgt}(g)} & z^2_{0,1} - az^2_{0,0} \\ \mathcal{K}^1_{\mathrm{Lft}(g)} - b \cdot \mathcal{K}^0_{\mathrm{Lft}(g)} & \mathcal{K}^0_{\mathrm{Rgt}(g)} - b \cdot \mathcal{K}^0_{\mathrm{Rgt}(g)} & z^2_{1,0} - az^2_{0,0} \\ \mathcal{K}^1_{\mathrm{Lft}(g)} - c \cdot \mathcal{K}^0_{\mathrm{Lft}(g)} & \mathcal{K}^1_{\mathrm{Rgt}(g)} - c \cdot \mathcal{K}^0_{\mathrm{Rgt}(g)} & z^2_{1,1} - az^2_{0,0} \end{bmatrix}$$

and $a = \mathcal{K}_g^{\mathrm{op}_g(0,1)}/\mathcal{K}_g^{\mathrm{op}_g(0,0)}$, $b = \mathcal{K}_g^{\mathrm{op}_g(1,0)}/\mathcal{K}_g^{\mathrm{op}_g(0,0)}$, $c = \mathcal{K}_g^{\mathrm{op}_g(1,1)}/\mathcal{K}_g^{\mathrm{op}_g(0,0)}$ If the determinant of $\mathcal{M}'$ is not zero, this will mean $B_g = 0, C_g = 0, D_g = 0$ along with $A_g = 0$. So, it would not satisfy the original gate equation. Hence, the only way A_g can be zero is if the determinant of $\mathcal{M}'$ is zero. The determinant of $\mathcal{M}'$ is a polynomial with 4 variables with maximum degree 6. To see that this polynomial is non-zero, notice that coefficient of the term $(z^2_{1,1}\mathcal{K}^1_{\mathrm{Lft}(g)}\mathcal{K}^1_{\mathrm{Rgt}(g)}$ is 1 unless a or b or c has a very specific value with probability of at most $3/q$. Hence, by the Schwartz Zippel lemma, the maximum probability of the determinant being zero is $6/q$

Hence, for a total of n gates, the maximum probability of the output being $\perp$ is $n(3/q + 1/q + 3/q + 6/q) = 13n/q$.

Unless $\perp$ is the output of the garbling algorithm Gb, the gate evaluation correct □

Theorem B.2. *The function* gateGb *has* GateSim *security in the random oracle model.*

Proof. The simulator is defined as follows -

- $\mathcal{S}^*(\mathcal{K}^t_{\mathrm{Lft}(g)}, \mathcal{K}_{\mathrm{Rgt}(g)u}\mathcal{K}_g^{T_0(t,u)}, g, \mathsf{par})$:
  (1) $\mathcal{K}^{1-t}_{\mathrm{LWire}(0)}, \mathcal{K}^{1-u}_{\mathrm{RWire}(0)}, O \leftarrow \mathbb{F}_q$
  (2) (A_g, B_g, C_g, D_g)
  $\leftarrow \mathsf{gateGb}(\mathsf{ConstantGate}, \mathcal{K}^t_{\mathrm{Lft}(g)}, \mathcal{K}^u_{\mathrm{Rgt}(g)}, \mathcal{K}^{1-t}_{\mathrm{Lft}(g)},$
  $\mathcal{K}^{1-u}_{\mathrm{Rgt}(g)}, \mathcal{K}_{g_0}^{T_0(t,u)}, g, \mathsf{par})$
  (3) Return (A_g, B_g, C_g, D_g)

Since $A_g \neq 0$, in both cases, each query to the random oracle requires guessing the inputs $\mathcal{K}^{1-t}_{\mathrm{Lft}(g)}, \mathcal{K}^{1-u}_{\mathrm{Rgt}(g)}$. This choice must be made without querying the random oracle. The only difference between the real and the simulated gate is that in case of the real gate, there random oracle is bound to satisty the polynomial equation at least for one choice of $\mathcal{K}^{1-t}_{\mathrm{Lft}(g)}, \mathcal{K}^{1-u}_{\mathrm{Rgt}(g)}$. Similiarly, for the constant gate, the the random oracle is bound to match for constant gate for at least one pair of input values. The adversary must spend at least one query to the random oracle to make any decision about the choice.

In case of the real gate, the probabily of choosing the specific values $\mathcal{K}^{1-t}_{\mathrm{Lft}(g)}, \mathcal{K}^{1-u}_{\mathrm{Rgt}(g)}$ that are guranteed to satisfy the equations is $1/(q-1)^2$. Similarly for the constant gate, the probability of finding the correct inputs to the random oracle is $1/(q-1)^2$. In all other cases, the probability of satisfying all the three equations for each query to the random oracle is $1/q^3$. Hence, the difference in probability for each query is at most $(2/(q-1)^2 - 1/q^3) \leq 2/(q-1)^2 \leq 4/q^2$ [Since $(q-1)^2 > q^2/2$ for $q \geq 4$].

Hence, for Q queries, the advantage of the adversary is $4Q/q^2$. □

C Proofs Omitted from Section 6

C.1 Proof of Lemma 6.5

In this section we present the full proof of Lemma 6.5. It may be helpful to review the proof sketch included along with the statement of the lemma before proceeding.

We start by proving the following lemma. For any group element $\mathbf{g}$ in some group $\mathbb{G}$ and $x \in \mathbb{F}_q$ where q is a prime number, we let $\mathbf{g}^x$ denote $\underbrace{\mathbf{g} \cdots \mathbf{g}}_{x \text{ times}}$ where x is treated as an integer.

Lemma C.1. *Let p be the prime order of the elliptic curve group $\mathbb{G}$ on the base field $\mathbb{F}_q$. Given $\mathbf{g} \in \mathbb{G}$, for uniformly random $x \leftarrow \mathbb{F}_q$ and $y \leftarrow \mathbb{F}_p$, the distribution of $\mathbf{g}^x$ is $O(1/\sqrt{q})$ far from the distribution of $\mathbf{g}^y$.*

Proof. Let h be the cofactor of the elliptic curve, so the number of points on the elliptic curve is hp. Hence, we have $|c| \leq 2\sqrt{q}$ where $c = q + 1 - hp$.

If c is positive, for any $x < c$, the number of $y < q$ such that $\mathbf{g}^y = \mathbf{g}^x$ is $\lceil q/p \rceil = h + 1$, and for $x \geq c$, the number of $y < q$ such that $\mathbf{g}^y = \mathbf{g}^x$ is $\lfloor q/p \rfloor = h$. Hence, the statistical difference can be

bounded as follows:

$$
\begin{aligned}
&\frac{1}{2}\sum_{x\in\{0..p-1\}}\left|\Pr_{y\leftarrow\mathbb{F}_q}[\mathrm{g}^y=\mathrm{g}^x]-\Pr_{y\leftarrow\mathbb{F}_p}[\mathrm{g}^y=\mathrm{g}^x]\right|\\
\le&\frac{1}{2}\sum_{x\in\{0..c-1\}}\left|\Pr_{y\leftarrow\mathbb{F}_q}[\mathrm{g}^y=\mathrm{g}^x]-\Pr_{y\leftarrow\mathbb{F}_p}[\mathrm{g}^y=\mathrm{g}^x]\right|\\
&+\frac{1}{2}\sum_{x\in\{c..p-1\}}\left|\Pr_{y\leftarrow\mathbb{F}_q}[\mathrm{g}^y=\mathrm{g}^x]-\Pr_{y\leftarrow\mathbb{F}_p}[\mathrm{g}^y=\mathrm{g}^x]\right|\\
=&\frac{1}{2}\sum_{x\in\{0..c-1\}}|(h+1)/q-1/p|+\sum_{x\in\{c..p-1\}}|h/q-1/p|\\
=&\frac{1}{2}\sum_{x\in\{0..c-1\}}|(h+1)/(hp+c-1)-1/p|\\
&+\sum_{x\in\{c..p-1\}}|h/(hp+c-1)-1/p|\\
=&\frac{1}{2}\sum_{x\in\{0..c-1\}}|((h+1)p-(hp+c-1))/p(hp+c-1)|\\
&+\sum_{x\in\{c..p-1\}}|(hp-(hp+c-1))/p(hp+c-1)|\\
=&\frac{1}{2}\sum_{x\in\{0..c-1\}}|(p-c+1)/p(hp+c-1)|\\
&+\sum_{x\in\{c..p-1\}}|-(c-1)/p(hp+c-1)|\\
=&\frac{1}{2}c(p-c+1)/p(hp+c-1)+(p-c)(c-1)/p(hp+c-1)\\
=&\frac{1}{2}(c(p-c+1)+(p-c)(c-1))/p(hp+c-1)\\
=&\frac{1}{2}(pc-c^2+c+pc-p-c^2+c)/p(hp+c-1)\\
=&\frac{1}{2}(2pc-2c^2+2c-p)/p(hp+c-1)\\
=&\frac{1}{2}(2pc-2c^2+2c-p)/pq\\
=&c/q-2c^2/pq+2c/pq-1/q\\
\le&c/q+2c/pq\\
=&c(p+1)/pq\\
\le&(2\sqrt{q})(p+1)/pq\\
=&(2/\sqrt{q})(1+1/p)\\
\le&4/\sqrt{q}
\end{aligned}
$$

On the other hand, if c is negative, for any $x \ge p+c$, the number of $y<q$ such that $\mathrm{g}^y=\mathrm{g}^x$ is $\lfloor q/p\rfloor-1=h-1$, and for $x<p-c$, the number of $y<q$ such that $\mathrm{g}^y=\mathrm{g}^x$ is $\lfloor q/p\rfloor=h$. Hence, the statistical difference is -

$$
\begin{aligned}
&\frac{1}{2}\sum_{x\in\{0..p-1\}}\left|\Pr_{y\leftarrow\mathbb{F}_q}[\mathrm{g}^y=\mathrm{g}^x]-\Pr_{y\leftarrow\mathbb{F}_p}[\mathrm{g}^y=\mathrm{g}^x]\right|\\
\le&\frac{1}{2}\sum_{x\in\{0..p-c-1\}}\left|\Pr_{y\leftarrow\mathbb{F}_q}[\mathrm{g}^y=\mathrm{g}^x]-\Pr_{y\leftarrow\mathbb{F}_p}[\mathrm{g}^y=\mathrm{g}^x]\right|\\
&+\frac{1}{2}\sum_{x\in\{p-c..p-1\}}\left|\Pr_{y\leftarrow\mathbb{F}_q}[\mathrm{g}^y=\mathrm{g}^x]-\Pr_{y\leftarrow\mathbb{F}_p}[\mathrm{g}^y=\mathrm{g}^x]\right|\\
=&\frac{1}{2}\sum_{x\in\{0..p-c-1\}}|h/q-1/p|+\sum_{x\in\{p-c..p-1\}}|(h-1)/q-1/p|\\
=&\frac{1}{2}\sum_{x\in\{0..p-c-1\}}|h/(hp+c-1)-1/p|\\
&+\sum_{x\in\{p-c..p-1\}}|(h-1)/(hp+c-1)-1/p|\\
=&\frac{1}{2}\sum_{x\in\{0..p-c-1\}}|(hp-(hp+c-1))/p(hp+c-1)|\\
&+\sum_{x\in\{p-c..p-1\}}|(hp-p-(hp+c-1))/p(hp+c-1)|\\
=&\frac{1}{2}\sum_{x\in\{0..p-c-1\}}|(-c+1)/p(hp+c-1)|\\
&+\sum_{x\in\{p-c..p-1\}}|(c-p-1)/p(hp+c-1)|\\
=&-\frac{1}{2}[(p-c)(c-1)/p(hp+c-1)+c(p-c+1)/p(hp+c-1)]\\
=&-\frac{1}{2}(pc-c^2-p+c+pc-c^2+c)/p(hp+c-1)\\
\le&-\frac{1}{2}(2pc+2c)/pq\\
=&-2c(1+1/p)/q\\
\le&-2c/q\\
\le&2(2\sqrt{q})/q\\
=&4/\sqrt{q}
\end{aligned}
$$

□

Lemma C.2. *Let the expression g^{xy} for $G\in\mathbb{G}$ represent $(\mathrm{g}^x)^y$. Then, for any PPT adversary $\mathcal{A}$,*

$$
\left|\Pr_{\substack{y\leftarrow\mathbb{F}_q\\x\leftarrow\mathbb{F}_p\\\mathrm{g}\leftarrow\mathbb{G}}}[\mathcal{A}(\mathrm{g},\mathrm{g}^x,\mathrm{g}^y,\mathrm{g}^{xy})=1]-\Pr_{\substack{x,y\leftarrow\mathbb{F}_p\\\mathrm{g}\leftarrow\mathbb{G}}}[\mathcal{A}(\mathrm{g},\mathrm{g}^x,\mathrm{g}^y,\mathrm{g}^{xy})=1]\right|\le\frac{4}{\sqrt{q}}
$$

Proof. If $x\in\mathbb{F}_p, y\in\mathbb{F}_q$, we have $(\mathrm{g}^y)^x=(\mathrm{g}^{y \bmod p})^x=(\mathrm{g}^{x(y \bmod p) \bmod p})=\mathrm{g}^{xy \bmod p}=(\mathrm{g}^x)^{y \bmod p}=\mathrm{g}^{xy}$. We can construct an adversary for distinguishing between $(\mathrm{g},\mathrm{g}^y)|y\leftarrow\mathbb{F}_q$ and $(\mathrm{g},\mathrm{g}^y)|y\leftarrow\mathbb{F}_p$ using such an adversary $\mathcal{A}$ as follows -

- Choose $x\leftarrow\mathbb{F}_p$

- Return $\mathcal{A}(\mathrm{g}, \mathrm{g}^x, \mathrm{g}^y, (\mathrm{g}^y)^x)$

The above adversary has the same advantage as $\mathcal{A}$. From lemma C.1, we know that this advantage is at most $4\sqrt{q}$. □

We define altDDH which is a variation of the DDH problem, but with one of the exponents comming from the base field of the curve, instead of the field modulo the order of the elliptic curve group.

*Definition C.3 (*altDDH*).* Given given $y \leftarrow \mathbb{F}_q; x, z \leftarrow \mathbb{F}_p$ and $\mathrm{g} \leftarrow \mathbb{G}$, the advatage $\mathrm{Adv}^{\mathrm{altDDH}}_{\mathbb{G},Q}$ is given by:

$$\mathrm{Adv}^{\mathrm{altDDH}}_{\mathbb{G},\mathcal{A}} = \left| \Pr_{\substack{\mathrm{g}\leftarrow\mathbb{G}\\ y\leftarrow\mathbb{F}_q\\ x,z\leftarrow\mathbb{F}_p}} [\mathcal{A}(\mathrm{g}, \mathrm{g}^x, \mathrm{g}^y, \mathrm{g}^{xy})] - \Pr_{\substack{\mathrm{g}\leftarrow\mathbb{G}\\ y\leftarrow\mathbb{F}_q\\ x,z\leftarrow\mathbb{F}_p}} [\mathcal{A}(\mathrm{g}, \mathrm{g}^x, \mathrm{g}^y, \mathrm{g}^z)] \right|$$

Lemma C.4. *Given given $y \leftarrow \mathbb{F}_q; x, z \leftarrow \mathbb{F}_p$ and $\mathrm{g} \leftarrow \mathbb{G}$, for every PPT adversary to distinguish between $(\mathrm{g}, \mathrm{g}^x, \mathrm{g}^y, \mathrm{g}^{xy})$ and $(\mathrm{g}, \mathrm{g}^x, \mathrm{g}^y, \mathrm{g}^z)$ with an advantage of $\mathrm{Adv}^{\mathrm{altDDH}}_{\mathbb{G},\mathcal{A}}$, there exists a PPT adversary for the DDH problem with and advantage of at least $\mathrm{Adv}^{\mathrm{altDDH}}_{\mathbb{G},\mathcal{A}} - 16/\sqrt{q}$*

Proof. Let the advantage of any adversary $\mathcal{A}$ be $\mathrm{Adv}^{\mathrm{altDDH}}_{\mathbb{G},\mathcal{A}}$ and the advantage of the same adversary for the DDH problem be $\mathrm{Adv}^{DDH}_{\mathbb{G},\mathcal{A}}$. Then -

$$\begin{aligned}
&\mathrm{Adv}^{\mathrm{altDDH}}_{\mathbb{G},\mathcal{A}} \\
&= \left| \Pr_{\substack{y\leftarrow\mathbb{F}_q\\ x\leftarrow\mathbb{F}_p\\ \mathrm{g}\leftarrow\mathbb{G}}} [\mathcal{A}(\mathrm{g}, \mathrm{g}^x, \mathrm{g}^y, \mathrm{g}^{xy}) = 1] - \Pr_{\substack{y\leftarrow\mathbb{F}_q\\ x,z\leftarrow\mathbb{F}_p\\ \mathrm{g}\leftarrow\mathbb{G}}} [\mathcal{A}(\mathrm{g}, \mathrm{g}^x, \mathrm{g}^y, \mathrm{g}^z) = 1] \right| \\
&\leq \left| \Pr_{\substack{y\leftarrow\mathbb{F}_q\\ x\leftarrow\mathbb{F}_p\\ \mathrm{g}\leftarrow\mathbb{G}}} [\mathcal{A}(\mathrm{g}, \mathrm{g}^x, \mathrm{g}^y, \mathrm{g}^{xy}) = 1] - \Pr_{\substack{x,y\leftarrow\mathbb{F}_p\\ \mathrm{g}\leftarrow\mathbb{G}}} [\mathcal{A}(\mathrm{g}, \mathrm{g}^x, \mathrm{g}^y, \mathrm{g}^{xy}) = 1] \right| \\
&\quad + \left| \Pr_{\substack{x,y\leftarrow\mathbb{F}_p\\ \mathrm{g}\leftarrow\mathbb{G}}} [\mathcal{A}(\mathrm{g}, \mathrm{g}^x, \mathrm{g}^y, \mathrm{g}^{xy}) = 1] - \Pr_{\substack{x,y,z\leftarrow\mathbb{F}_p\\ \mathrm{g}\leftarrow\mathbb{G}}} [\mathcal{A}(\mathrm{g}, \mathrm{g}^x, \mathrm{g}^y, \mathrm{g}^z) = 1] \right| \\
&\quad + \left| \Pr_{\substack{x,y,z\leftarrow\mathbb{F}_p\\ \mathrm{g}\leftarrow\mathbb{G}}} [\mathcal{A}(\mathrm{g}, \mathrm{g}^x, \mathrm{g}^y, \mathrm{g}^z) = 1] - \Pr_{\substack{y\leftarrow\mathbb{F}_q\\ x,z\leftarrow\mathbb{F}_p\\ \mathrm{g}\leftarrow\mathbb{G}}} [\mathcal{A}(\mathrm{g}, \mathrm{g}^x, \mathrm{g}^y, \mathrm{g}^z) = 1] \right| \\
&\leq 4/\sqrt{q} + \mathrm{Adv}^{DDH}_{\mathbb{G},\mathcal{A}} + 4\sqrt{q} \\
&= 8/\sqrt{q} + \mathrm{Adv}^{DDH}_{\mathbb{G},\mathcal{A}}
\end{aligned}$$

Hence, $\mathrm{Adv}^{DDH}_{\mathbb{G},\mathcal{A}} \geq \mathrm{Adv}^{\mathrm{altDDH}}_{\mathbb{G},\mathcal{A}} - 16/\sqrt{q}$ □

Note: The altDDH problem can also be written as the problem of distinguishing between $(\mathrm{g}_1, \mathrm{g}_2, \mathrm{g}_1^y, \mathrm{g}_2^y)$ and $(\mathrm{g}_1, \mathrm{g}_2, \mathrm{g}_1^y, \mathbf{h})$ where $\mathrm{g}_1, \mathrm{g}_2, \mathbf{h} \leftarrow \mathbb{G}$ and $y \leftarrow \mathbb{F}_q$.

Lemma C.5. *Given $y \leftarrow \mathbb{F}_q$ and $\mathrm{g}_1, \mathrm{g}_2, ..., \mathrm{g}_n, \mathbf{h} \leftarrow \mathbb{G}$, ff the advantage of a PPT adversary to distinguish between $(\mathrm{g}_1, \mathrm{g}_2, ..., \mathrm{g}_n, \mathrm{g}_1^y, \mathrm{g}_2^y, ..., \mathrm{g}_n^y)$ and $(\mathrm{g}_1, \mathrm{g}_2, ..., \mathrm{g}_n, \mathrm{g}_1^y, \mathrm{g}_2^y, ..., \mathrm{g}_{n-1}^y, \mathbf{h})$ is ϵ, then there exists a PPT adversary that solves the* altDDH *problem with the same advantage.*

Proof. Let us assume that there is an adversary $\mathcal{A}$ that can distinguish with an advantage ϵ. We make the following adversary for distinguishing between $(\mathrm{g}_1, \mathrm{g}_1^y, \mathrm{g}_n, \mathrm{g}_n^y)$ and $(\mathrm{g}_1, \mathrm{g}_1^y, \mathrm{g}_n, \mathbf{h})$. Given the input
$(\mathrm{g}_1, \mathrm{g}_2, ..., \mathrm{g}_n, \mathrm{g}_1^y, \mathrm{g}_2^y, ..., \mathrm{g}_{n-1}^y, \mathbf{h}')$,

(1) Choose $x_2, x_2, ..., x_{n-1} \leftarrow \mathbb{F}_p$.
(2) Compute $\mathrm{g}_i = \mathrm{g}_1^{x_i}$ for each $i \in \{2, 3, ..., n-1\}$.
(3) Compute $\mathrm{g}_i^y = (\mathrm{g}_1^y)^{x_i}$ for each $i \in \{2, 3, ..., n-1\}$.
(4) Return $\mathcal{A}(\mathrm{g}_1, \mathrm{g}_2, ..., \mathrm{g}_n, \mathrm{g}_1^y, \mathrm{g}_2^y, ..., \mathrm{g}_{n-1}^y, \mathbf{h})$

Now, when $\mathbf{h}' = \mathrm{g}_n^y$, the argument passed is $(\mathrm{g}_1, \mathrm{g}_2, ..., \mathrm{g}_n, \mathrm{g}_1^y, \mathrm{g}_2^y, ..., \mathrm{g}_n^y)$. On the other hand, when $\mathbf{h}' = \mathbf{h}$, the argument passed is $(\mathrm{g}_1, \mathrm{g}_2, ..., \mathrm{g}_n, \mathrm{g}_1^y, \mathrm{g}_2^y, ..., \mathrm{g}_{n-1}^y, \mathbf{h})$. Hence, the above adversary also has the same advantage. □

Lemma C.6. *Given $y \leftarrow \mathbb{F}_q$ and $\mathrm{g}_1, \mathrm{g}_2, ..., \mathrm{g}_n, \mathbf{h}_1, \mathbf{h}_2 \leftarrow \mathbb{G}$, if the advantage of a PPT adversary to distinguish between $(\mathrm{g}_1, \mathrm{g}_2, ..., \mathrm{g}_n, \mathrm{g}_1^y, \mathrm{g}_2^y, ..., \mathrm{g}_n^y)$ and $(\mathrm{g}_1, \mathrm{g}_2, ..., \mathrm{g}_n, \mathrm{g}_1^y, \mathrm{g}_2^y, ..., \mathrm{g}_{n-2}^y, \mathbf{h}_1, \mathbf{h}_1)$ is ϵ, then there exists an adversary that solves the* altDDH *problem with an advantage of at least $\epsilon/2$.*

Proof. For any adversary $\mathcal{A}$, the advantage

$$\begin{aligned}
\mathrm{Adv}^{\mathrm{altDDHL}}_{\mathbb{G},\mathcal{A}} &= |\Pr[\mathcal{A}(\mathrm{g}_1, \mathrm{g}_2, ..., \mathrm{g}_n, \mathrm{g}_1^y, \mathrm{g}_2^y, ..., \mathrm{g}_{n-1}^y, \mathrm{g}_n^y) = 1] \\
&\quad - \Pr[\mathcal{A}(\mathrm{g}_1, \mathrm{g}_2, ..., \mathrm{g}_n, \mathrm{g}_1^y, \mathrm{g}_2^y, ..., \mathrm{g}_{n-2}^y, \mathbf{h}_1, \mathbf{h}_2) = 1]| \\
&\leq |\Pr[\mathcal{A}(\mathrm{g}_1, \mathrm{g}_2, ..., \mathrm{g}_n, \mathrm{g}_1^y, \mathrm{g}_2^y, ..., \mathrm{g}_{n-1}^y, \mathrm{g}_n^y) = 1] \\
&\quad - \Pr[\mathcal{A}(\mathrm{g}_1, \mathrm{g}_2, ..., \mathrm{g}_n, \mathrm{g}_1^y, \mathrm{g}_2^y, ..., \mathrm{g}_{n-2}^y, \mathrm{g}_{n-1}^y, \mathbf{h}_2) = 1]| \\
&\quad + |\Pr[\mathcal{A}(\mathrm{g}_1, \mathrm{g}_2, ..., \mathrm{g}_n, \mathrm{g}_1^y, \mathrm{g}_2^y, ..., \mathrm{g}_{n-1}^y, \mathbf{h}_2) = 1] \\
&\quad - \Pr[\mathcal{A}(\mathrm{g}_1, \mathrm{g}_2, ..., \mathrm{g}_n, \mathrm{g}_1^y, \mathrm{g}_2^y, ..., \mathrm{g}_{n-2}^y, \mathbf{h}_1, \mathbf{h}_2) = 1]| \\
&= \mathrm{Adv}^{\mathrm{altDDH}}_{\mathbb{G},\mathcal{A}} + \mathrm{Adv}^{\mathrm{altDDH}}_{\mathbb{G},\mathcal{A}} \\
&= 2\mathrm{Adv}^{\mathrm{altDDH}}_{\mathbb{G},\mathcal{A}}
\end{aligned}$$

□

Now we are ready to finish the proof of Lemma 6.5.

Proof of lemma 6.5. Let

$$\bar{\mathbf{e}} = ([\mathrm{g}_i]_{i\in[n]}, [\mathbf{h}_i]_{i\in[n]}, [\mathrm{g}_i^x]_{i\in[n-1]}, [\mathbf{h}_i^x]_{i\in[n-1]}, [\mathrm{g}_i^y]_{i\in[n-1]}, [\mathbf{h}_i^y]_{i\in[n-1]})$$

Now,

$$\begin{aligned}
\mathrm{Adv}^{\mathrm{altDDH}^*}_{\mathbb{G},\mathcal{A}} &= |\Pr[\mathcal{A}(\bar{\mathbf{e}}, \mathrm{g}_n^x, \mathrm{g}_n^y, \mathbf{h}_n^x, \mathbf{h}_n^y] - \Pr[\mathcal{A}(\bar{\mathbf{e}}, \mathrm{g}_1', \mathrm{g}_2', \mathbf{h}_1', \mathbf{h}_2')]| \\
&\leq |\Pr[\mathcal{A}(\bar{\mathbf{e}}, \mathrm{g}_n^x, \mathrm{g}_n^y, \mathbf{h}_n^x, \mathbf{h}_n^y] - \Pr[\mathcal{A}](\bar{\mathbf{e}}, \mathrm{g}_n^x, \mathrm{g}_2', \mathbf{h}_n^x, \mathbf{h}_2')| \\
&\quad + |\Pr[\mathcal{A}(\bar{\mathbf{e}}, \mathrm{g}_n^x, \mathrm{g}_2', \mathbf{h}_n^x, \mathbf{h}_2')] - \Pr[\mathcal{A}(\bar{\mathbf{e}}, \mathrm{g}_1', \mathrm{g}_2', \mathbf{h}_1', \mathbf{h}_2')]| \\
&= \mathrm{Adv}^{\mathrm{altDDHL}}_{\mathbb{G},\mathcal{A}} + \mathrm{Adv}^{\mathrm{altDDHL}}_{\mathbb{G},\mathcal{A}} \\
&= 2\mathrm{Adv}Adv^{\mathrm{altDDHL}}_{\mathbb{G},\mathcal{A}}
\end{aligned}$$

Hence, $\mathrm{Adv}^{\mathrm{altDDHL}}_{\mathbb{G},\mathcal{A}} \leq 2\mathrm{Adv}^{\mathrm{altDDH}^*}_{\mathbb{G},\mathcal{A}} \leq 4\mathrm{Adv}^{\mathrm{altDDH}}_{\mathbb{G},\mathcal{A}}$
$\leq 4\mathrm{Adv}^{DDH}_{\mathbb{G},\mathcal{A}} + 64/\sqrt{q}$ due to Lemma C.6 and Lemma C.4. □

C.2 Proof of Lemma 6.6

Lemma C.7. *If the probability of collision for a distribution $\mathcal{D}$ over a set $\mathcal{S}$ is $Col(\mathcal{D})$, i.e. $Col(\mathcal{D}) = \Pr_{x,y\leftarrow\mathcal{D}}[x = y]$, then:*

$$\sum_{y\in\mathcal{S}} \left| \Pr_{x\leftarrow\mathcal{D}}[x = y] - \Pr_{x\leftarrow\mathcal{U}}[x = y] \right| \leq \sqrt{|\mathcal{S}|Col(\mathcal{D}) - 1}$$

where $\mathcal{U}$ is the uniform distribution over $\mathcal{S}$.

Proof.

$$\begin{aligned}
&\sum_{y\in\mathcal{S}} (\Pr[x = y|x \leftarrow \mathcal{D}] - \Pr[x = y|x \leftarrow \mathcal{U}])^2 \\
&= \sum_{y\in\mathcal{S}} \Pr[x = y|x \leftarrow \mathcal{D}]^2 + \sum_{y\in\mathcal{S}} \Pr[x = y|x \leftarrow \mathcal{U}]^2 \\
&\qquad - 2\sum_{y\in\mathcal{S}} \Pr[x = y|x \leftarrow \mathcal{D}] \Pr[x = y|x \leftarrow \mathcal{U}] \\
&= Col(\mathcal{D}) + 1/|\mathcal{S}| - 2\sum_{y\in\mathcal{S}} \Pr[x = y|x \leftarrow \mathcal{D}] \Pr[x = y|x \leftarrow \mathcal{U}] \\
&= Col(\mathcal{D}) + 1/|\mathcal{S}| - 2\sum_{y\in\mathcal{S}} \Pr[x = y|x \leftarrow \mathcal{D}] \cdot (1/|\mathcal{S}|) \\
&= Col(\mathcal{D}) + 1/|\mathcal{S}| - 2(1/|\mathcal{S}|) \cdot \sum_{y\in\mathcal{S}} \Pr[x = y|x \leftarrow \mathcal{D}] \\
&= Col(\mathcal{D}) + 1/|\mathcal{S}| - 2(1/|\mathcal{S}|) \cdot 1 \\
&= Col(\mathcal{D}) - 1/|\mathcal{S}|
\end{aligned}$$

Now, using the general inequality $|v|_1 \leq \sqrt{|\mathcal{S}|}|v|_2$ for a vector v of dimension $|\mathcal{S}|$, we have

$$\begin{aligned}
&\sum_{y\in\mathcal{S}} |\Pr[x = y|x \leftarrow \mathcal{D}] - \Pr[x = y|x \leftarrow \mathcal{U}]| \\
&\leq \sqrt{|\mathcal{S}|}\sqrt{\sum_{y\in\mathcal{S}} (\Pr[x = y|x \leftarrow \mathcal{D}] - \Pr[x = y|x \leftarrow \mathcal{U}])^2} \\
&= \sqrt{|\mathcal{S}|}\sqrt{Col(\mathcal{D}) - 1/|\mathcal{S}|} \\
&= \sqrt{|\mathcal{S}|Col(\mathcal{D}) - 1}
\end{aligned}$$

□

Lemma C.8. *Let $\mathbf{M}'$ be a matrix just like $\mathbf{M}$, but r_0, s_0, t_0, u_0 replaced by r'_0, s'_0, t'_0, u'_0 respectively. Note that the fourth rows of both $\mathbf{M}$ and $\mathbf{M}'$ are the same. Let $\mathbf{M}''$ be a matrix created by $\mathbf{M}$ on top of $\mathbf{M}'$ where $\mathbf{M}'$ is placed upside down, and one copy of the common row removed, i.e.,*

$$\mathbf{M}'' = \begin{bmatrix} M''_1 & M''_2 & M''_3, \end{bmatrix}$$

where

$$M''_1 = \begin{bmatrix} r_0 & r_0^2 & s_0 & s_0^2 \\ r_0 & r_0^2 & s_0 & s_0^2 \\ r_1 & r_1^2 & s_1 & s_1^2 \\ r_1 & r_1^2 & s_1 & s_1^2 \\ r_1 & r_1^2 & s_1 & s_1^2 \\ r'_0 & r_0'^2 & s'_0 & s_0'^2 \\ r'_0 & r_0'^2 & s'_0 & s_0'^2 \end{bmatrix}$$

$$M''_2 = \begin{bmatrix} t_0 & t_0^2 & u_0 & u_0^2 \\ t_1 & t_1^2 & u_1 & u_1^2 \\ t_0 & t_0^2 & u_0 & u_0^2 \\ t_1 & t_1^2 & u_1 & u_1^2 \\ t'_0 & t_0'^2 & u'_0 & u_0'^2 \\ t_1 & t_1^2 & u_1 & u_1^2 \\ t'_0 & t_0'^2 & u'_0 & u_0'^2 \end{bmatrix}$$

$$M''_3 = \begin{bmatrix} r_0t_0 & r_0^2t_0^2 & s_0u_0 & s_0^2u_0^2 \\ r_0t_1 & r_0^2t_1^2 & s_0u_1 & s_0^2u_1^2 \\ r_1t_0 & r_1^2t_0^2 & s_1u_0 & s_1^2u_0^2 \\ r_1t_1 & r_1^2t_1^2 & s_1u_1 & s_1^2u_1^2 \\ r_1t'_0 & r_1^2t_0'^2 & s_1u'_0 & s_1^2u_0'^2 \\ r'_0t_1 & r_0'^2t_1^2 & s'_0u_1 & s_0'^2u_1^2 \\ r'_0t'_0 & r_0'^2t_0'^2 & s'_0u'_0 & s_0'^2u_0'^2 \end{bmatrix}$$

where $r_1, s_1, t_1, u_1 \in \mathbb{F}_q \setminus \{0\}$ and $r_0, s_0, t_0, u_0, r'_0, s'_0, t'_0, u'_0 \leftarrow \mathcal{I}$ and $\mathcal{I} \subset \mathbb{F}_q \setminus \{0\}$. Then, the maximum probability of the rank of M being at most 6 is at most $58/|I|^2$, and the probability of the rank being less than 5 is at most $120/|\mathcal{I}|^4$.

Proof. Let $\hat{r_0} = r_0 - r_1, \hat{s_0} = s_0 - s_1, \hat{t_0} = t_0 - t_1, \hat{u_0} = u_0 - u_1, \hat{\hat{r_0}} = r_0^2 - r_1^2, \hat{\hat{s_0}} = s_0^2 - s_1^2, \hat{\hat{t_0}} = y_1^2 - t_1^2, \hat{\hat{u_0}} = y_2^2 - u_1^2$ and $\hat{r'_0} = r'_0 - r_1, \hat{s'_0} = s'_0 - s_1, \hat{t'_0} = t'_0 - t_1, \hat{u'_0} = u'_0 - u_1, \hat{\hat{r'_0}} = r_0'^2 - r_1^2, \hat{\hat{s'_0}} = s_0'^2 - s_1^2, \hat{\hat{t'_0}} = t_0'^2 - u_1^2, \hat{\hat{u'_0}} = u_0'^2 - t_1^2$. Performing the following row and column operations - $R_3 \leftarrow R_3 - R_4$, $R_1 \leftarrow R_1 - R_4$ and $R_2 \leftarrow R_2 - R_4$, $R_1 \leftarrow R_1 - R_3 - R_2$, $C_9 \leftarrow C_9 - r_1C_5 - t_1C_1$, $C_{10} \leftarrow C_{10} - r_1^2C_6 - t_1^2C_2$,$C_{11} \leftarrow C_{11} - s_1C_7 - u_1C_3$, $C_{10} \leftarrow C_{10} - s_1^2C_8 - u_1^2C_4$, we have the new matrix $\mathbf{M}''' = \begin{bmatrix} M'''_1 & M'''_2 \end{bmatrix}$ where

$$M'''_1 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \hat{r_0} & \hat{\hat{r_0}} & \hat{s_0} & \hat{\hat{s_0}} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \hat{t_0} & \hat{\hat{t_0}} & \hat{u_0} & \hat{\hat{u_0}} \\ r_1 & r_1^2 & s_1 & s_1^2 & t_1 & t_1^2 & u_1 & u_1^2 \\ 0 & 0 & 0 & 0 & \hat{t'_0} & \hat{\hat{t'_0}} & \hat{u'_0} & \hat{\hat{u'_0}} \\ \hat{r'_0} & \hat{\hat{r'_0}} & \hat{s'_0} & \hat{\hat{s'_0}} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$M'''_2 = \begin{bmatrix} \hat{r_0}\hat{t_0} & \hat{\hat{r_0}}\hat{\hat{t_0}} & \hat{s_0}\hat{u_0} & \hat{\hat{s_0}}\hat{\hat{u_0}} \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ -r_1t_1 & -r_1^2t_1^2 & -s_1u_1 & -s_1^2u_1^2 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ \hat{r'_0}\hat{t'_0} & \hat{\hat{r'_0}}\hat{\hat{t'_0}} & \hat{s'_0}\hat{u'_0} & \hat{\hat{s'_0}}\hat{\hat{u'_0}} \end{bmatrix}$$

For the matrix to not be full-rank, there must be some $\alpha, \gamma, \beta, \delta, \beta', \gamma', \alpha'$ such that $\alpha R_1 + \beta R_2 + \gamma R_3 + \delta R_4 + \beta' R_6 + \gamma' R_5 + \alpha' R_7 = 0$. We consider the following cases:

(1) $\delta = 0$: In this case, either $\alpha \neq 0 \wedge \alpha' \neq 0$ [Due to $C9$], or $\gamma \neq 0 \wedge \gamma' \neq 0$ [Due to C_5] or $\beta \neq 0 \wedge \beta' \neq 0$ [Due to C_1]. We consider $\beta \neq 0 \wedge \beta' \neq 0$ below, since the other cases are similar. In this case, we have the following equations:

$$\hat{r_0}/\hat{r'_0} = \hat{\hat{r}_0}/\hat{\hat{r'_0}} = \hat{s_0}/\hat{s'_0} = \hat{\hat{s}_0}/\hat{\hat{s'_0}} \tag{6}$$

Simplifying, we have the following three equations:

$$\hat{r_0}(\hat{r'_0}^2 + 2r_1\hat{r'_0}) = \hat{r'_0}(\hat{r_0}^2 + 2r_1\hat{r_0}) \tag{7}$$

$$\hat{s_0}(\hat{s'_0}^2 + 2r_1\hat{s'_0}) = \hat{s'_0}(\hat{s_0}^2 + 2r_1\hat{s_0}) \tag{8}$$

They are two polynomial equations of maximum degree 3 with no shared variables. Hence, the maximum probability of both satisfying is $9/|\mathcal{I}|^2$. Similarly, the maximum probability when $\gamma \neq 0 \wedge \gamma' \neq 0$ is $9/|\mathcal{I}|^2$ and the maximum probability when $\beta \neq 0 \wedge \beta' \neq 0$ is $36/|\mathcal{I}|^2$.

(2) $\delta \neq 0$: In this case, we assume $\delta = 1$ without loss of generality. Now, from C_1, we find that at least one of β, β' is not zero. We consider the following cases:

(a) $\beta \neq 0 \wedge \beta' = 0$: In this case, from C_1, C_2, C_3, C_4, we have the following equations -

$$\hat{r_0}/r_1 = \hat{\hat{r}_0}/r_1^2 \tag{9}$$

$$\hat{s_0}/s_1 = \hat{\hat{s}_0}/s_1^2 \tag{10}$$

$$\tag{11}$$

Simplifying, we get $\hat{r_0} = \hat{s_0} = -r_1$. Hence, the probability of this condition being satisfied is $1/|\mathcal{I}|^2$.

(b) $\beta' \neq 0 \wedge \beta = 0$ This is symmetric to the previous case.

(c) $\beta \neq 0 \wedge \beta' \neq 0$: From C_1, C_2, we have the following equations:

$$\beta\hat{r_0} + r_1 + \beta'\hat{r'_0} = 0 \tag{12}$$

$$\beta\hat{\hat{r}_0} + r_1 + \beta'\hat{\hat{r'_0}} = 0 \tag{13}$$

Solving for β, β', we get the following:

$$\beta = \frac{r_1(\hat{r'_0} - \hat{\hat{r'_0}})}{\hat{r_0}\hat{\hat{r'_0}} - \hat{r'_0}\hat{\hat{r}_0}} \tag{14}$$

$$\beta' = \frac{r_1(\hat{r_0} - \hat{\hat{r}_0})}{\hat{r'_0}\hat{\hat{r}_0} - \hat{r_0}\hat{\hat{r'_0}}} \tag{15}$$

Now, the probability that the denominator being zero is at most $3/|\mathcal{I}|$ since it is a non-zero polynomial of maximum degree 3. Otherwise, from C_3 we have the equation:

$$\beta\hat{s_0} + s_1 + \beta'\hat{s'_0} = 0 \tag{16}$$

with no shared variable with the value of β, β thus computed. Hence, the maximum probability of this being satisfied is $1/|\mathcal{I}|$. With symmetric reasoning with columns C_5, C_6, C_7, we find that the maximum probability of the corresponding equations being zero is $2/|\mathcal{I}|$. Since there are no variables in common between C_1, C_2, C_3 and C_5, C_6, C_7, the maximum probability that both sets of equations are satisfied is at most $4/|\mathcal{I}|^2$.

Hence, the maximum probability of the matrix not being full-rank is at most $58/|\mathcal{I}|^2$. Now, consider the probability of the rank being at most 5. In this case, there must be some $\alpha, \gamma, \beta, \beta', \gamma', \alpha'$ such that $\alpha R_1 + \beta R_2 + \gamma R_3 + \gamma' R_5 + \beta' R_6 + \alpha' R_7 = 0$. We consider the following cases -

(1) either $\alpha \neq 0 \wedge \alpha' \neq 0$ [Due to $C9$], or $\gamma \neq 0 \wedge \gamma' \neq 0$ [Due to C_5] or $\beta \neq 0 \wedge \beta' \neq 0$ [Due to C_1]. We consider $\beta \neq 0 \wedge \beta' \neq 0$ below, since the other cases are similar. In this case, we have the following equations:

$$\hat{r_0}/\hat{r'_0} = \hat{\hat{r}_0}/\hat{\hat{r'_0}} = \hat{s_0}/\hat{s'_0} = \hat{\hat{s}_0}/\hat{\hat{s'_0}} \tag{17}$$

Simplifying, we have the following two equations:

$$\hat{r_0}(\hat{r'_0}^2 + 2r_1\hat{r'_0}) = \hat{r'_0}(\hat{r_0}^2 + 2r_1\hat{r_0}) \tag{18}$$

$$\hat{s_0}(\hat{s'_0}^2 + 2r_1\hat{s'_0}) = \hat{s'_0}(\hat{s_0}^2 + 2r_1\hat{s_0}) \tag{19}$$

They are two polynomial equations of maximum degree 3 with no shared variables. Hence, the maximum probability of both satisfying is $9/|\mathcal{I}|^2$ [Note that there is another equation that would also have to be satisfied, but that only reduces the probability of satisfaction]. Similarly, the maximum probability when $\gamma \neq 0 \wedge \gamma' \neq 0$ is $9/|\mathcal{I}|^2$ and the maximum probability when $\beta \neq 0 \wedge \beta' \neq 0$ is $36/|\mathcal{I}|^2$. Now, we consider the following cases:

(a) $\beta \neq 0 \wedge \beta' \neq 0$: In this case, there must be some $\alpha_{x'}, \beta_{x'}, \gamma_{x'}, \delta_{x'}, \gamma'_{x'}, \alpha'_{x'}$ such that $\alpha_{x'} r_1 + \beta_{x'} s_1 + \gamma_{x'} R_2 + \delta_{x'} R_4 + \gamma'_{x'} R_5 + \alpha'_{x'} R_7 = 0$. We consider the following cases:

(i) $\delta_{x'} = 0$: In this case $\gamma_{x'} = 0$, so we consider the following cases -

(A) $\beta_{x'} \neq 0 \wedge \gamma'_{x'} = 0$: This case is symmetric to the previous case when considering y instead of x.

(B) $\gamma'_{x'} \neq 0 \wedge \beta_{x'} = 0$: This case is also symmetric with the previous case.

(C) $\beta_{x'} \neq 0 \wedge \gamma'_{x'} \neq 0$: The maximum probability that this case being satisfied is at most $1/|\mathcal{I}|$ by the same argument as 2c.

(D) $\gamma_{x'} \neq 0 \vee \gamma'_{x'} \neq 0$: by an argument similar to that in the previous case, we can conclude that the maximum probability that this case being satisfied is $36/|\mathcal{I}|^2$.

(ii) $\delta_{x'} = 1$: In this case, $\gamma_{x'} \neq 0$. Hence, we have the following equations.

$$\hat{r_0}/r_1 = \hat{\hat{r}_0}/r_1^2 \tag{20}$$

$$\hat{s_0}/s_1 = \hat{\hat{s}_0}/s_1^2 \tag{21}$$

$$\tag{22}$$

These are two independent multivariate equations with maximum degree 2. Hence, the maximum probability of both being satisfied is $4/|\mathcal{I}|^2$.

We can have a similar argument for the case where $\alpha \neq 0 \wedge \alpha' \neq 0$, or $\gamma \neq 0 \wedge \gamma' \neq 0$.

Hence, the maximum probability of the rank being at most 5 is at most $120/|\mathcal{I}|^4$. □

Lemma C.9. *Let $\mathcal{S}_1$ and $\mathcal{S}_2$ be two finite sets. Then -*

$$\Pr_{\substack{r_0 \leftarrow \mathcal{S}_1 \\ s_0 \leftarrow \mathcal{S}_2}}[r_0 = s_0] = \frac{|\mathcal{S}_1 \cap \mathcal{S}_2|}{|\mathcal{S}_1||\mathcal{S}_2|}$$

Proof.

$$\begin{aligned}
&\Pr[r_0 = s_0] \\
=&\Pr_{r_0 \in \mathcal{S}_1 \cap \mathcal{S}_2}[r_0 = s_0] \Pr[r_0 \in \mathcal{S}_1 \cap \mathcal{S}_2] \\
&\quad + \Pr_{r_0 \notin \mathcal{S}_1 \cap \mathcal{S}_2}[r_0 = s_0] \Pr[r_0 \notin \mathcal{S}_1 \cap \mathcal{S}_2] \\
=&\Pr_{r_0 \in \mathcal{S}_1 \cap \mathcal{S}_2}[r_0 = s_0] \Pr[r_0 \in \mathcal{S}_1 \cap \mathcal{S}_2] + 0 \\
=&\Pr_{r_0 \in \mathcal{S}_1 \cap \mathcal{S}_2}[r_0 = s_0] \frac{|\mathcal{S}_1 \cap \mathcal{S}_2|}{|\mathcal{S}_1|} \\
=&\frac{1}{|\mathcal{S}_2|} \frac{|\mathcal{S}_1 \cap \mathcal{S}_2|}{|\mathcal{S}_1|} \\
=&\frac{|\mathcal{S}_1 \cap \mathcal{S}_2|}{|\mathcal{S}_1||\mathcal{S}_2|}
\end{aligned}$$

□

Proof of Lemma 6.6. Without loss of generality, we let $a^* = 1, b^* = 1$, since reordering the rows of a matrix only reorders the rows of product matrix. Now, let r_0, s_0, t_0, u_0 and r'_0, s'_0, t'_0, u'_0 be the randomly chosen values for the first and the second pick respectively. Now, let the outputs of the two picks be $(a_1, a_2, ..., a_{12})$ and $(a'_1, a'_2, ..., a'_{12})$ respectively. Also let $\mathbf{M}'$ be a matrix just like $\mathbf{M}$, but r_0, s_0, t_0, u_0 replaced by r'_0, s'_0, t'_0, u'_0 respectively; i.e. -

$$M'_1 = \begin{bmatrix} r'_0 & {r'_0}^2 & s'_0 & {s'_0}^2 \\ r'_0 & {r'_0}^2 & s'_0 & {s'_0}^2 \\ r_1 & r_1^2 & s_1 & s_1^2 \\ r_1 & r_1^2 & s_1 & s_1^2 \end{bmatrix}$$

$$M'_2 = \begin{bmatrix} t'_0 & {t'_0}^2 & u'_0 & {u'_0}^2 \\ t_1 & t_1^2 & u_1 & u_1^2 \\ t'_0 & {t'_0}^2 & u'_0 & {u'_0}^2 \\ t_1 & t_1^2 & u_1 & u_1^2 \end{bmatrix}$$

$$M'_3 = \begin{bmatrix} r'_0 t'_0 & {r'_0}^2 {t'_0}^2 & s'_0 u'_0 & {s'_0}^2 {u'_0}^2 \\ r'_0 t_1 & {r'_0}^2 t_1^2 & s'_0 u_1 & {s'_0}^2 u_1^2 \\ r_1 t'_0 & r_1^2 {t'_0}^2 & s_1 u'_0 & s_1^2 {u'_0}^2 \\ r_1 t_1 & r_1^2 t_1^2 & s_1 u_1 & s_1^2 u_1^2 \end{bmatrix}$$

$$\mathbf{M}' = \begin{bmatrix} M'_1 & M'_2 & M'_3 \end{bmatrix}$$

Let $\mathcal{S}_1$ be the set of all the values of $(a_1, a_2, ..., a_{12})$ such that

$$\begin{bmatrix} \alpha_{0,0} \\ \alpha_{0,1} \\ \alpha_{1,0} \\ \alpha_{1,1} \end{bmatrix} = \mathbf{M} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{12} \end{bmatrix}$$

and $\mathcal{S}_2$ be all the values of $(a_1, a_2, ..., a_{12})$ such that

$$\begin{bmatrix} \alpha_{0,0} \\ \alpha_{0,1} \\ \alpha_{1,0} \\ \alpha_{1,1} \end{bmatrix} = \mathbf{M}' \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{12} \end{bmatrix}$$

Hence, $\mathcal{S}_1 \cap \mathcal{S}_2$ is all the values that satisfy the following equation:

$$\begin{bmatrix} \alpha_{0,0} \\ \alpha_{0,1} \\ \alpha_{1,0} \\ \alpha_{1,1} \\ \alpha_{1,0} \\ \alpha_{0,1} \\ \alpha_{0,0} \end{bmatrix} = \mathbf{M}'' \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{12} \end{bmatrix}$$

Now, given any specific values of $r_0, s_0, t_0, u_0, r'_0, s'_0, t'_0, u'_0$, the probability of collision is equal to $\frac{|\mathcal{S}_1 \cap \mathcal{S}_2|}{|\mathcal{S}_1||\mathcal{S}_2|}$. Now, let us consider the size of $|\mathcal{S}_1 \cap \mathcal{S}_2|$. Let v_1, v_2 be two members of $\mathcal{S}_1 \cap \mathcal{S}_2$. Then $Mv_1 = Mv_2 \Rightarrow M(v_1 - v_2) = 0$. Hence, $v_1 - v_2$ must be in the null space of M. On the other hand, let v_3 be a vector in the nullspace of M and v_1 is a solution, then $M(v_1 + v_3) = Mv_1 + Mv_3 = Mv_1$.Hence, if v_1 is in $\mathcal{S}_1 \cap \mathcal{S}_2$ and v_3 is in the null space of M, $v_1 + v_3$ is also in $\mathcal{S}_1 \cap \mathcal{S}_2$. Hence, the size of $\mathcal{S}_1 \cap \mathcal{S}_2$ is the same as the size of the null space of M if $\mathcal{S}_1 \cap \mathcal{S}_2$ is not empty. By the same logic, the size of $\mathcal{S}_1$ is the null space is the size of M_1 and the size of $\mathcal{S}_2$ is the size of the null space of M_2. Now, the way the distribution is chosen, the size of the null space of both M_1 and M_2 is q^8. And the size of the null space of M is $q^{12-rank(M)}$. Hence, for given values of $r_0, s_0, t_0, u_0, r'_0, s'_0, t'_0, u'_0$, the probability of collision is at most $\frac{q^{12-rank(M)}}{q^{16}} = \frac{1}{q^{4+rank(M)}}$.

Now, the probability -

$$\begin{aligned}
&\Pr[(a_1, a_2, ..., a_{12}) = (a'_1, a'_2, ..., a'_{12})] \\
=&\Pr[(a_1, a_2, ..., a_{12}) = (a'_1, a'_2, ..., a'_{12}) | rank(M) = 7] \Pr[rank(M) = 7] \\
&+ \Pr[(a_1, a_2, ..., a_{12}) = (a'_1, a'_2, ..., a'_{12}) | rank(M) = 6] \Pr[rank(M) = 6] \\
&+ \Pr[(a_1, a_2, ..., a_{12}) = (a'_1, a'_2, ..., a'_{12}) | rank(M) = 5] \Pr[rank(M) = 5] \\
&+ \Pr[(a_1, a_2, ..., a_{12}) = (a'_1, a'_2, ..., a'_{12}) | rank(M) = 4] \Pr[rank(M) = 4] \\
&+ \Pr[(a_1, a_2, ..., a_{12}) = (a'_1, a'_2, ..., a'_{12}) | rank(M) < 4] \Pr[rank(M) < 4] \\
\leq& 1/q^{11} + (1/q^{10}) \cdot 58/|\mathcal{I}|^2 + (1/q^9) \cdot 120/|\mathcal{I}|^4 + (1/q^8) \cdot 120/|\mathcal{I}|^4 + 0 \\
\leq& 1/q^{11} + (1/q^{10}) \cdot 58/|\mathcal{I}|^2 + (1/q^8) \cdot 240/|\mathcal{I}|^4 + 0
\end{aligned}$$

Now, if $q/|\mathcal{I}| = h'$, we have the bound on the probability -

$$\begin{aligned}
&1/q^{11} + 58{h'}^2/q^{12} + 240{h'}^4/q^{12} \\
=&1/q^{11} + (58{h'}^2 + 240{h'}^4)/q^{12}
\end{aligned}$$

Hence, the statistical difference between $\mathcal{D}$ and $\mathcal{U}$ is at most

$$\begin{aligned}&\frac{1}{2}\sqrt{q^{11}(1/q^{11}+(58h'^2+240h'^4)/q^{12})-1}\\&\quad=\frac{1}{2}\sqrt{(58h'^2+240h'^4)/q}\leq\frac{1}{2}\sqrt{298h'^4/q}\\&\quad\leq\frac{1}{2}(18h'^2)/\sqrt{q}=\frac{9h'^2}{\sqrt{q}}\end{aligned}$$

□

D Efficiency of AES based construction

We can use lookup gates to optimize AES evaluation. Note that AES requires arithmetic operations on a 8-bit binary field and also looking up an S-box that is non-algebraic in nature. Without lookup gates, this will require complete bit-splitting of every label to work. But, with lookup tabels, we can implement XOR gates between bytes by listing the output for every possible input. We can also use lookup tables for S-box lookup. We can keep the labels split into bytes until we reach the output.

We make a comparison of the number of arithmetic gates in the garbled circuit proof of correctness per gate of the circuit f that we are garbling. We make a comparison against AES-based systems, random-oracle-based schemes with both SHA-256 and Poseidon-2. We notice that random oracle-based schemes require one bit of information in the wire-labels to tell the evaluator which encrypted labels to decrypt. Extraction of a bit requires a large number of gates in the zero-knowledge proof system, so we can use quadratic residuosity in instead. The quadratic residuosity of a label can be shown by providing the square root as a secret witness, and non-residuosity can be proved by showing the square root of the label multiplied by a known fixed non-residue.

For AES, we use the optimization ideas from [18]. AES involves the following steps: (1) Substitution, (2) Shift-rows, (3) Mix-columns, and (4) Add-round-key.

The step shift-rows does not take any gates and can be swapped in order with substitution. We can also use lookups to do the substitution and mix-columns at the same time. The Mix-columns multiplies the value matrix with a fixed matrix. Matrix multiplication happens column-wise, so, we can simply store this multiplication as a table for each possible value of the 4-byte column. However, this would require a table of size 2^32. Instead, we can use two different tables of each 16-bit - one for the upper half of a column and one for the lower half of the column. Moreover, instead of looking up the column of the input values, we can modify the table to lookup the column after applying the S-box for each of the bytes in each half-column. This eliminates any extra lookup for S-box. Since, there are 8 half-columns, this can be done with 8 table lookups. This step also needs to XOR the 4-byte columns obtained from the lookups of the half-columns. Adding the round keys requires 16 more XOR-lookups. In total, we need $8 + 16 + 16 = 40$ lookups per round of AES for this. Since AES has 10 rounds and an extra key-XOR in the beginning, the total number of look-up gates for one AES computation is $40 \times 10 + 16 = 416$.

We now have to consider the key-scheduling algorithm. AES key-scheduling algorithm requires one XOR for most bytes except two, the first and the last byte of the round key. For the first byte, it needs one more XOR and one more substituion; and the last byte requires just one additional XOR. So, we need $16 + 2 + 1 = 19$ XOR gates for the key-scheduling that must be done 10 times, once per round. So, the number of lookups needed for key-scheduling is $10 \times 19 = 190$.

Since we need a total of 8 AES computations per garbled gate, the total number of gates required to each garbled gate using AES is $(416 + 190) \times 8 = 4848$.

We must also note that doing the search for mixed-columns requires a 4-byte output. Since we need to keep the bytes separate, $1 + 2 + 4 = 7$ wires are required. We need one wire for table ID, 2 for two input wires, and 4 for 4 output wires.